

Resource Adaptive Automated Task Scheduling Using Deep Deterministic Policy Gradient In Fog Computing

PRASHANTH CHOPPARA¹, S Sudheer Mangalampalli² (Member, IEEE)

¹School of Computer Science and Engineering, VIT-AP University, Amaravati, Andhra Pradesh 522237 India (e-mail: prashanth.deva.777@gmail.com)

²Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, Karnataka, India

Corresponding author: S Sudheer Mangalampalli (e-mail: ms.sudheer@manipal.edu)

ABSTRACT The rapidly increasing complexity of Internet of Things applications and the exponential growth in data generation pose significant challenges in terms of latency and network capacity constraints, especially in cloud computing. Fog computing carries have emerged as an effective solution by decentralizing data processing, reducing latency, and bringing computation closer to the data sources. This paper presents a novel adaptive scheduling framework based on the DDPG algorithm for task scheduling optimization in fog computing environments. Our framework is based on DDPG, a reinforcement learning algorithm well suited for continuous action spaces, adapting scheduling strategies to real-time changes in task demands and resource availability. Such capability allows for highly accurate decisions to be made in real time, which is of particular importance for latency-sensitive applications such as autonomous vehicles, remote healthcare, and industrial automation. The contributions of this paper include the development of an adaptive scheduling framework that can support sequential, parallel, and dependency-based scheduling. This framework improves several critical performance metrics by 30%, reduces makespan, reduces fault tolerance by 25%, and improves system scalability and reliability by 20%. Data processing localization through our approach reduces bandwidth and latency usage by up to 40% and improves data privacy and efficiency in data management. Simply simulated this work using Google Cloud Jobs datasets, and the results support the promising combination of advanced machine learning and fog computing. This research demonstrates the transformative impact of deep reinforcement learning in improving task scheduling in distributed computing environments, laying a strong foundation for further research aimed at harnessing the full capabilities of fog computing for the ongoing demands of IoT applications.

INDEX TERMS Fog Computing, Task Scheduling, Deep Deterministic Policy Gradient (DDPG), Internet of Things (IoT).

I. INTRODUCTION

Cloud computing has revolutionized the way data is stored, processed, and managed in recent years. It provides a scalable, centralized model for resource-intensive tasks, similar to other resources used in various fields [1]. It relies on large data centers that provide significant computing power and storage at a very low cost. However, cloud computing has some limitations, which are related to real-time data processing and latency-sensitive applications that demand responsiveness. Centralized cloud data centers are usually geographically separated from edge devices, which introduces latency [2]. This latency affects applications such as autonomous vehicles, remote healthcare, and industrial automation. Sensitive information being sent over these networks

creates data in IoT systems, increasing network bandwidth and security and privacy [3] [4]. These limitations suggest the need for an alternative model that moves computation closer to the data source, to reduce latency and improve data privacy.

Fog computing has emerged as a powerful complement to cloud computing and addresses challenges by distributing data processing and bringing computational resources closer to where the data is generated [5]. By placing resources at the edge, fog computing reduces the constant transmission of data to cloud servers located in remote locations on another network, resulting in low latency and bandwidth reservation [6]. The architecture supports applications that require immediate response, improves privacy levels through

localized processing of data, and provides real-time capabilities for decision-making. Despite the advantage of fog computing, there are some new challenges as the resources of the nodes are limited, requiring effective management to optimize system performance [7], [8]. Compared to a cloud data center that can use almost unlimited resource pools, fog nodes operate in a highly constrained environment, so task scheduling must effectively achieve good response to high resource utilization and system stability [9].

Scheduling in fog computing is a difficult multi-objective problem. Specifically, it involves choosing the best order of available resources and assigning tasks under dynamic conditions [10]. To efficiently schedule tasks on fog nodes, one must balance conflicting goals such as maximizing the scaling of different task loads, minimizing the time required to execute critical tasks, and increasing fault tolerance and reliability. Energy consumption is controlled to extend node lifetime, and a wide range of tasks must be executed in a fog computing environment [11]. These tasks differ in priority, inter-dependencies, and resource requirements. Some tasks depend on each other's execution results, and some tasks can be executed concurrently or must be executed in a specific order to improve reliability. The heterogeneity and interdependence of tasks require an adaptive scheduling strategy that can adapt to changing demands in real time based on resource availability and task dependencies [12]. A dynamic environment with limited resources makes traditional rule-based and static scheduling approaches ineffective, resulting in high energy consumption and sub-optimal performance.

Reinforcement learning has emerged as a particularly promising approach to efficiently apply dynamic and adaptive scheduling in complex computing environments. In such a context, we consider task scheduling optimization in the field of fog computing integrated model RL Deep Deterministic Policy Gradient Algorithm [13]. This algorithm is well suited for this scenario because it is a continuous action space algorithm that allows the system to make fine-grained, real-time scheduling decisions. DDPG, unlike other static scheduling algorithms, can adapt to changing environmental conditions. These adaptive scheduling policies are developed through feedback from past actions [14]. DDPG is particularly well suited for task scheduling in fog computing due to its ability to operate in continuous action spaces. Unlike discrete-action RL algorithms, DDPG enables fine-grained decision making by mapping states to continuous actions, which is critical for real-time resource allocation and adapting to rapid changes in task demands. DDPG actor-critic architecture ensures robust policy learning, with the actor network generating optimal actions and the critic network evaluating these actions to improve learning efficiency. This combination makes DDPG an ideal choice for developing a dynamic and adaptive scheduling framework that can optimize critical performance metrics such as makespan, fault tolerance, scalability, and energy efficiency. In this paper, the author presents an intelligent scheduling framework using DDPG in tasks, which are mainly classified into three

types of scheduling: sequential, parallel scheduling, and dependency-based scheduling. Each fog node is scheduled for different task requirements.

Sequential, parallel, and dependency-based scheduling to meet the diverse needs of fog computing environments. Sequential scheduling ensures that tasks are executed in a strict order, reducing latency and makespan, which is critical for time-sensitive applications such as medical monitoring and emergency response, where fault tolerance and predictable execution are critical. Parallel scheduling increases resource utilization by distributing independent tasks across multiple fog nodes for simultaneous execution, improving scalability and reducing energy waste. This approach is ideal for handling high task volumes, such as in data processing and analytics, without sacrificing responsiveness. Dependency-based scheduling manages tasks with inter-dependencies, ensuring that they are executed in the correct order, preserving accuracy and reliability. This is especially important for mission-critical applications such as scientific simulations and autonomous systems, where the integrity of task execution directly affects the results. Together, these strategies enable the framework to effectively handle diverse workloads while optimizing key performance metrics in dynamic fog environments.

A. MOTIVATION AND BACKGROUND

The exponential growth of Internet of Things (IoT) applications has created new challenges in computing systems, particularly concerning real-time data processing, latency reduction, and data privacy. Cloud computing, while a powerful paradigm, often fails to meet the stringent requirements of latency-sensitive and privacy-critical applications due to its centralized architecture and physical distance from end devices. The inherent delays and potential security risks make cloud computing unsuitable for applications such as autonomous vehicles, remote healthcare, and industrial automation. Fog computing has emerged as a complementary model to cloud computing, addressing these challenges by bringing computational resources closer to data sources. By localizing data processing, fog computing significantly reduces latency minimizes bandwidth usage, and enhances data privacy by reducing the need to transfer sensitive information to centralized cloud servers. The distributed and resource-constrained nature of fog environments introduces new challenges for task scheduling, requiring strategies that can dynamically adapt to workload fluctuations and resource availability while optimizing multiple performance metrics such as makespan, fault tolerance, and energy consumption.

The paper contributions the field of task scheduling in fog computing using the Deep Deterministic Policy Gradient (DDPG) algorithm are as follows:

- In this paper presents an adaptive scheduling framework built on the DDPG algorithm, enabling dynamic task allocation based on real-time variations in workload and resource availability.

- The framework adopts the three distinct scheduling strategies sequential, parallel, and dependency-based scheduling to the unique requirements of different task types in fog computing environments.
- The processing tasks closer to data sources, the framework reduces the need for extensive data transmission.
- The DDPG algorithm optimizes metrics including makespan, fault tolerance, scalability, reliability, and energy efficiency.
- The framework scales to handle the workloads, from small-scale to large-scale task distributions, maintaining efficiency and responsiveness in highly dynamic fog environments.

The remaining paper organized to Section II Related Work, Section III Proposed Methodology, Section IV Proposed Framework and Section V Experimental results are organized as follows.

II. RELATED WORK

In author has proposed a new architecture, the Double Blockchain-Enabled Architecture, in [15], to address the huge challenges in response times of real-time IoT tasks in energy consumption optimizations in cloud-fog collaborative networks. A new scheduling model presented here to optimize both energy consumption and deadlines along with two blockchain-enabled algorithms priority-aware algorithm, or DBRP accelerated ant colony system algorithm, or DBAACS. Their comprehensive evaluation on several parameters showed that their approach superior compared baseline in system makespan, energy efficiency, task completion rate, and deadline satisfaction. This work adds a comprehensive framework for scheduling tasks in fog computing and significantly moves the state of the art in blockchain-based IoT task management.

In [16] present an efficient load balancing designed to optimize resource scheduling of cloud-based communication for healthcare environments. The authors use reinforcement learning algorithms including Genetic Algorithm (GA), SARSA to effectively improve resource management. This approach addresses the need to manage the vast amount of data processed by smart medical sensors of compromising performance metrics such as throughput and latency. The proposed framework is also significant in terms of energy efficiency, reduced make-span, and reduced latency, bringing forward a very promising approach for managing large-scale data in healthcare systems. Simulations were conducted with MATLAB, validating the performance of the framework using metrics latency

In [17], the focus is on the challenge of optimizing scheduling in heterogeneous fog computing. They propose a multi-objective approach that balances cost awareness with performance metrics such as latency and computational efficiency. The research introduces novel algorithms that leverage fog computing capabilities to efficiently distribute bag-of-task applications. These tasks are characterized by their independence and varying computational demands. Using

a cost-aware strategy, the paper aims to optimize resource utilization and operational costs, which are critical in fog computing scenarios where resources are decentralized and potentially limited.

In [18], the author proposes integrating blockchain technology into fog and cloud computing frameworks to improve task scheduling. It proposes an life-choice-based optimization algorithm with the aim of optimizing key performance metrics such as power, latency, and cost in FCB. In summary, this algorithm for ILCO emphasizes acceleration in convergence speed and managing population diversity for resource management in distributed computing systems. The authors address the high computational demands and complexity from blockchain operations in fog computing. Their work is a significant contribution to the ongoing efforts on balancing optimization in performance and resource utilization, efficiently managing task scheduling in IoT environments. However, this research provides bridging methods between theoretical optimization techniques and practical applications in fog-cloud architectures to build a robust framework for future advancements in IoT and blockchain integrations.

In [19], the task scheduling challenge of CPSS is proposed in the fog computing framework. They develop a multi-objective scheduling method to simultaneously optimize several key performance indicators, including computational efficiency, resource allocation, and system responsiveness. Building on the uniqueness of the fog computing model, such as proximity to data sources and the ability to implement distributed processing, new algorithms are proposed based on cyber and social demand considerations with respect to the constraints in physical systems. Not only are the performance characteristics improved by CPSS, but social aspects are also adequately addressed to ensure the system's ability to satisfy user demands under dynamic conditions.

In [20] the author presents the optimization and allocation of resources in CFC systems. An advanced version of MAO, a bio-inspired metaheuristic, is derived from the complex problems presented in task scheduling of highly dynamic environments. The algorithm leverages the excellent reproductive abilities of axolotls to improve task scheduling processes by optimizing the allocation of computing tasks, which improves resource utilization, reduces latency, and increases operational efficiency in cloud and fog computing ecosystems.

In [21], the author and colleagues explore resource allocation optimization in multi-tier computing networks (MTCN), which are fundamental to the efficient management of intelligent transportation systems (ITS). The authors present a dynamic resource scheduling algorithm that is sensitive to multiple objectives such as computing, storage, bandwidth, and latency, facilitated by a deep reinforcement learning-based network. This approach significantly improves the flexibility and efficiency of resource utilization in cloud, fog, and edge layers, thereby overcoming the heterogeneity and high-intensity dynamics inherent in such environments.

Authors in [22] propose a novel multi-objective simulated

annealing algorithm to improve task scheduling in their paper Multi-Objective Task Scheduling in Cloud Programming Approaching It focuses on dealing with two competing requirements, namely service time and service cost minimization, along with the security requirements of task execution in an IoT setting. Using GPA, it provides a balanced framework between the conflicting objectives of the objectives: one based on execution time and service provision cost and the other based on controlling the access levels serving different clients that require execution of tasks across distributed nodes.

In the paper A New Operator in Multi-Objective Gravitational Search algorithm published by [23], the author proposes a new operator called star-quake to improve the multi-objective gravitational search (MOGSA) for IoT-based scheduling in a cloud-fog computing system. This improvement helps avoiding local optima and improving convergence towards optimal solutions by balancing selection pressure, exploitation, and exploration. The modified algorithm, called SQMOGSA, was tested against several multi-objective optimization methods and showed greater results in terms of reduction in makepan, energy consumption, and processing costs for tasks in a fog-cloud system.

In [24] authors discuss the need to address complexities associated with task offloading in fog computing environments, with an emphasis placed on improvement in the quality of service or QoS. A multi-agent deep reinforcement learning (DRL) framework to optimize task offloading decisions, which considers multiple QoS factors, such as end-to-end delay, energy consumption, task deadline, and priority, is proposed by the authors. The researchers modeled the offloading problem as a Markov Decision Process and implemented three model-free, off-policy DRL algorithms: Deep Queue-Network (DQN), Deep Deterministic Policy Gradient (DDPG), and Soft Actor-Critic (SAC). They applied these mechanisms to maximize the reward based on resource utilization while efficiently managing the demands faced by the fog computing infrastructure. Their proposed mechanisms were confirmed to be highly efficient and effective through extensive experiments. The results demonstrate that their approach can meet the deadlines of 96.23% of tasks at improved resource utilization. The contribution made by this study is indeed significant, as it proposes a very strong solution for optimizing task offloading in fog environments, which is particularly useful for applications with stringent QoS requirements.

In [25], a resource scheduling advance in cloud-fog computing delay-sensitive applications using an improved Gray Wolf optimization algorithm. This hybrid approach combines HEFT with the Gray Wolf optimization algorithm to minimize makepan and improve throughput. The algorithm performs optimization in both the exploration and exploitation phases to avoid local optimization. Experimental evaluation using iFogSim shows up to 72.56% improvement in throughput and significant reductions in makepan compared to traditional algorithms such as AEOSA, HHO, and PSO.

In [26], present the EEIoMT framework for real-time

task scheduling in healthcare IoT systems, addressing the challenges of latency and energy consumption. Using ECG sensors and fog nodes, the framework prioritizes critical tasks while maintaining efficient energy consumption. Simulations in iFogSim2 reveal superior performance in latency reduction and energy efficiency when compared to the CHTM and FNPA models. This work demonstrates its potential for continuous health monitoring in smart cities.

An effective dynamic load balancing (EDLB) algorithm is designed in [27] with a combination of CNN-based classifiers and modified particle swarm optimization (MPSO). This algorithm improves scheduling due to dynamic classification of fog nodes with optimized task assignments. Experimentation on healthcare applications has resulted in reduced response time and peak resource utilization. Experimental validation proves that EDLB outperforms traditional methods in load balancing performance and reduces makepan.

The author introduces an intelligent healthcare system that utilizes fog computing and deep learning for real-time health monitoring in [28]. The proposed system is optimized for low latency and resource utilization using IoT and FogBus integration. It is designed for cardiac health monitoring. The proposed system improves throughput and accuracy, ensuring energy efficiency. Simulation results clearly show its feasibility to achieve QoS and provide better patient outcomes.

Proposes a relief strategy in [29] A learning based strategy for delegation fault-tolerant fog systems. The algorithm optimizes reliability by dynamically delegating primary and replacement tasks based on machine learning. ReLIEF reduces task failure rates and workload asymmetries with up to 84% improvement in real-time applications.

Develop a new multi-objective directed acyclic graph (DAG) task scheduling approach (TSSAC) for fog computing environment with container migration to enhance system performance in [30]. Critical issues such as energy efficiency, load balancing, and makepan reduction in the Industrial Internet of Things (IIoT) are addressed in this study. By using DAGs to model tasks, the framework determines the priority order of subtasks and their optimal allocation to fog nodes using an advanced ant colony optimization (ACO) algorithm. The proposed method introduces a dynamic container migration mechanism to mitigate resource issues. Overload and underutilization. The mechanism adjusts task scheduling based on real-time server states, which ensures efficient energy consumption and balanced workload distribution. Simulations results shown by TSSAC framework got a 23.5% reduction in energy consumption compared to traditional ACO, with significant improves in load balancing and task completion time. This work presents a new approach by fine-tuning the pheromone and evaporation coefficients in ACO. Thus, the exploration and exploitation phases in it are efficient. Another feature used here is the container migration mechanism, which dynamically shifts tasks in a real-time fog computing system towards its optimization.

In [31], the author proposes an opposition-based chemical

reaction optimization (OBCR) algorithm to overcome the scheduling challenges in fog networks. Fog computing, as a decentralized infrastructure, often suffers from NP-complete task scheduling problems related to its dynamic and heterogeneous nature. It utilizes three important ingredients, the heuristic upward ranking technique CRO framework, and opposition-based learning in solving this approach. The former improves the fast search to identify the first feasible reasonable solutions; the latter increases the diversity and convergence without searching for local optima by the OBL algorithm. The four on-wall operators employed in the OBCR operators include decomposition, intermolecular inefficiency collision, and synthesis. These operators balance exploration and exploitation in the solution space, thereby optimizing task allocation. Extensive simulations performed using iFogSim demonstrate the improved performance of OBCR compared to techniques ACO, CEGA, and MOSSO. The algorithm improves the stability of the makespan and processing devices by up to 28%, thus showing efficacy for both small-scale and complex workflows such as montage and SIPHT. The proposed framework also exhibits significant robustness under varying levels of heterogeneity.

In author [32] presents a new task scheduling method by improving the artificial hummingbird algorithm. Their proposed approach is called CODA: Chaos, Opposition-Based Learning, and Differential Evolution Algorithm. CODA improves AHA with chaotic maps and opposition-based learning (OBL) for better initial population generation, and then it uses DE for optimization refinement. The CODA algorithm is applied to task scheduling, which aims to minimize energy consumption, preparation-span, and processing cost while maintaining the quality of service, QoS. The authors used AHP to rank tasks by their length and memory requirements. With such a ranking, CODA efficiently executes high-priority tasks. Extensive simulation results on 50 benchmark functions and comparisons with other popular meta-heuristic algorithms PSO, MFO and SSA prove the superiority of CODA. The proposed algorithm achieves an average of 46% reduction in makespan, 8% in cost and 41% in energy consumption compared to alternatives. CODA adaptive mechanisms effectively balance exploration and exploitation, avoiding local optima and achieving fast convergence.

In [33] author presents a two-stage optimal (2-ST) designed smart home environments using fog computing infrastructures. The proposed framework addresses various key challenges in a decentralized the fog nodes. such as latency, energy consumption, and network congestion, which are critical for delay-sensitive applications. This is a hybrid optimization approach using an ACO and PSO combination to prioritize tasks for the best possible assignment, where initially a naive Bayes machine learning model will be integrated into the system in order to select priorities in the first place. Delay-sensitive tasks in the system must be executed at their own pace. Such tasks are examples of smoke detectors and water leak detection systems. The authors used the iFogSim simulator to evaluate their performance. They

showed how the 2-ST technique greatly reduces latency, energy consumption, and network utilization compared to traditional methods. The proposed framework improves resource utilization by fog nodes, providing better quality of service toward IoT systems for smart home applications.

In [34], the author proposes a PSO task scheduling in fog computing for big data applications. This approach is made possible by using the advantages of GA for exploration and PSO for exploitation in effectively reducing execution, response time, and task completion time while maintaining resource utilization and energy efficiency. This hybrid model addresses the most important challenges in fog computing, including dynamic workload, resource heterogeneity, and scalability. The GA-PSO algorithm greatly enhances the efficiency of task scheduling by dynamically adapting to fog environments while optimizing computational and network resources. It was simulated by iFogSim simulator and evaluated in different configurations. Compared with traditional methods such as GA, PSO, and Particle Wavelet Optimization Algorithm (PWOA), the hybrid GA-PSO exhibits superior performance. It reduces execution time by up to 85.68%, response time by up to 75.40%, and task completion time by up to 85.23% in various cases. The Pareto optimization of the algorithm ensures a balanced trade-off between conflicting objectives such as energy and latency. The authors highlight flexibility GA-PSO algorithm in diverse fog environments and its potential in real-time applications including smart cities and IoT systems.

The author introduces the multi-objective Gray Wolf Optimizer (MGWO) algorithm designed for task scheduling problems cloud-fog computing environments [35]. The MGWO algorithm aims to reduce energy consumption and transmission delay by using efficient resource utilization and task assignment. It recognizes the limitations of meta-heuristic algorithms, such as poor avoidance of local optima and high computational complexity, and attempts to overcome these difficulties through the hierarchical decision-making structure of the gray wolf pack. In this regard, the MGWO algorithm uses several extensions, including a Pareto dominance approach to balance conflicting objectives and an external archive that ensures diversity by storing non-dominated solutions. The authors mathematically model the task scheduling problem using queuing theory, thereby demonstrating that the proposed algorithm is flexible enough to accommodate heterogeneous resources such as fog nodes and cloud servers. Extensive simulations conducted with MATLAB R2018b reveal that MGWO, NSGA-II and MPSO in significantly reducing energy consumption and delay for various workloads.

In [36] author addresses task scheduling in cloud-fog computing using a newly proposed algorithm the Multiobjective Harris Hawkes Optimization-Based Task Scheduling Algorithm, MoHHOTS, designed by Asad Ali et al. It combines the features of HHO with the proven ability of convergence and equilibrium between exploration and exploitation phases along with Pareto-optimal solutions to control the trade-off in terms of energy and delay. MoHHOTS has ability to adapt

itself to workload changes and dynamic network conditions. Therefore, the task scheduling solution it provides in IoT applications is scalable and energy efficient. The complexity of cloud-fog systems IoT devices take different forms and they create different computational requirements. Therefore, efficient processing of tasks is crucial for QoS. MoHHOTS utilizes a mathematical model that optimally allocates tasks across fog and cloud nodes with reduced latency, minimal energy consumption, and balanced resource utilization. Through MATLAB simulations, the algorithm is evaluated against state-of-the-art benchmarks such as MOGWO and the Cloud-Fog Cooperation Algorithm. The results show that MoHHOTS can achieve up to 25% improvement in terms of energy efficiency and makespan reduction.

In [37] author proposed a multi-objective hybrid marine predators algorithm for task scheduling in the fog computing environment. Their aim is to enable optimization with multiple conflicting objectives such as makepan, energy consumption, flow time, and carbon dioxide emission. MHMPA incorporates a polynomial mutation mechanism that enhances exploration capabilities and solution diversity. Further, using Lévy flight strategy it employs Cauchy distribution over the Gaussian distribution, strengthening exploitation capabilities and avoiding getting stuck in local optima. It produces solutions using Pareto optimality. Finally, wide experiments exhibit its superiority against MPPA and can be compared to state of art algorithms NSGA-II, SMPSO, OMOPSO, NSGA-III. Simulation results confirm that MHMPA achieves significant improvements concerning energy efficiency, makepan, and flow time and therefore is a robust approach, suitable for highly dynamic and diverse fog computing environments.

In [38]author formulated a Secure and Traceable Multi-key Image Retrieval in Cloud-Assisted Internet of Things (STMIR) provides a privacy-preserving image retrieval framework for IoT environments. STMIR improves retrieval accuracy using convolutional neural networks (CNNs) for robust image feature extraction and employs a novel privacy-preserving Mahalanobis distance comparison method based on Learning with Errors (LWE). The scheme addresses the key challenges of shared keys between users and traceability of malicious users through encrypted watermarks. By allowing multikey setup and ensuring high security, STMIR provides secure, efficient, and accurate image retrieval, outperforming existing methods in real-world tests.

In [39]author designed Efficient Privacy-Preserving Spatial Data Query in Cloud Computing presents a secure method for handling the outsourcing of spatial data queries on cloud servers, addressing issues related to high computational overhead and insecurity in encryption using two schemes PSDQ. In the former scheme, data privacy is ensured through a unified index structure, and in the latter, a GR-tree is included for faster sublinear query complexity through. The experiments conducted are very extensive, showing improvements in query efficiency and security. The system is well-suited for applications such as smart cities and

spatial data services.

In [40]author proposed a Electronic Well being eHealth frameworks can outsource information such as understanding eHealth records to versatile cloud servers, which we formally demonstrate for productivity, for example reducing nearby capacity and computational costs. Such a move could lead to protection suggestions near semi-honest cloud servers. Searchable encryption SE could encourage privacy-preserving forms based on catchphrases for encrypted information stored in the mobile cloud but most existing SE arrangements do not reinforce the temporal gap to control. The part that gives consent to clients for suggested time periods. So in this paper we plan a form of time controllable catchphrase that conspires by using a feature-based comparable gap to control. This enables clients to compare scrambled lists over suggested time periods. At that point, we step into a basic system that supports productive client refutation using secret sharing. We formally demonstrate the security of our proposed systems against selected keyword attack and key collaboration attack. In addition to achieving catchphrase mystery. We also evaluate the implementation of our proposed approach by using a real-world dataset to illustrate their general utility.

In [41]author formulated the Deadline-Aware and Load-Balanced Task Scheduling for Internet of Things in Fog Computing focuses on task scheduling for latency-sensitive IoT applications in fog computing environments. The DALB-Fog scheduling mechanism is introduced to reduce delays, balance loads across distributed fog nodes, and optimize resource utilization. A dual-scheduler system is adopted that prioritizes tasks according to deadlines and assigns them to fog nodes with low load and minimum completion time. This approach ensures that IoT applications meet their strict timing requirements, while avoiding over- or under-utilization of resources. Experimental evaluations reveal that DALB-Fog achieves superior average resource utilization, shorter response times, and reduced task execution times compared to contemporary scheduling methods, making it an effective solution for managing IoT workloads in fog computing systems.

In [42]author proposed a Workflow Scheduling Based on Asynchronous Advantage Actor-Critical Algorithm in Multi-Cloud Environment addresses the challenges faced by workflow scheduling in MCEs, characterized by complexity related to resource heterogeneity, dynamic QoS requirements, and task dependencies. Therefore, a novel scheduling algorithm, called MCWS-A3C, is proposed based on the A3C reinforcement learning technique used to optimize makespan, cost, and resource utilization. By modeling each cloud provider as an agent, the algorithm enables localized decision making and better adaptability to dynamic environments. A custom critical network is designed to handle real-world scientific workflow data, and a multi-objective reward function ensures a balance between different optimization objectives. Experimental results show that the MCWS-A3C algorithm significantly outperforms traditional

methods, achieving cost reductions of up to 55.12% and improved efficiency across multiple workflows.

In [43] author done a comprehensive review of task scheduling methods in cloud computing environments, meta-heuristics algorithms. Cloud computing is a form of resource virtualization and dynamic provisioning that requires efficient task scheduling to optimize performance and costs. This paper categorizes task scheduling objectives into user-centric (make-span, cost, and meeting deadlines) and provider-centric (load balancing, resource utilization, and energy efficiency). The review includes various meta-heuristic algorithms such as genetic algorithms (GAs), particle swarm optimization (PSO), ant colony optimization (ACO), and bio-inspired methods such as memetic algorithms and lion optimization. These approaches address the NP-complete nature of task scheduling by providing near-optimal solutions. This paper presents a classification of these methods and compares them based on their suitability for different scheduling objectives. The review highlights previously overlooked costs associated with storage, failure rate, and reliability, which have been found to be challenges in previous work. Future research directions focus on integrating advanced optimization methods, managing dynamic and heterogeneous cloud environments, and addressing emerging concerns such as energy efficiency and data security. The review article serves as a resource for selecting appropriate scheduling strategies in cloud computing.

III. PROPOSED METHODOLOGY

This methodology refers to tasks that are scheduled and classified into different classes based on the framework developed in this approach in a fog computing environment. The proposed framework, which is adjusted for scalability, reliability, and energy efficiency, adjusts for the dynamic nature and distributed characteristics of the fog. It starts with task prioritization, where task evaluation is based on key parameters including duration, resource requirements, deadlines, dependency counts, and impact factors. All these parameters are combined into a priority score, which allows the scheduler to execute more important tasks. Critical and time-sensitive activities are executed on time.

After prioritization, tasks are classified into three classes for execution to increase management efficiency. For sequential tasks, if one task depends on executing before another, an execution order is required, and such sequential execution is best suited for a series of stages such as data pipes. However, parallel tasks are tasks that can be executed on any number of virtual machines spread across the nodes of the fog, increasing throughput with minimal time taken to complete the tasks. Dependency-based tasks have dependencies; therefore, some operations are performed only after a specific requirement is met, and to maintain logical consistency in workflows that have complex relationships.

This method involves a task scheduler that matches prioritized and classified tasks with the available resources of fog nodes. It dynamically allocates tasks based on node avail-

ability, load, and suitability for the task type. For sequential tasks, the scheduler establishes a strict execution order by giving one task to the least loaded VM after another. For parallel tasks, it attempts to distribute the workload across multiple VMs with balanced resource usage. The scheduler checks the readiness of tasks; for dependency-based tasks, it instructs the scheduler to schedule dependent tasks only after all required tasks have completed.

A. TASK PRIORITY

In fog computing environments, tasks often have different requirements and limited computational resources, and the prioritization step serves as the foundation for efficient scheduling. By evaluating tasks on multiple criteria, each task is assigned a priority score P_i , which reflects its urgency and importance. This score ensures that the most curious tasks are scheduled first, improving system performance and resource utilization. The prioritization framework considers five key factors: task duration, resource requirements, deadline proximity, dependency count, and impact factor. Each of these factors is mathematically integrated to calculate P_i . General formula for priority score

$$P_i = w_1 \cdot \frac{1}{D_i} + w_2 \cdot \frac{1}{R_i} + w_3 \cdot \frac{1}{t_{\text{deadline},i}} + w_4 \cdot \frac{1}{|D_i| + 1} + w_5 \cdot I_i \quad (1)$$

Here, w_1, w_2, w_3, w_4 , and w_5 are normalized weights, summing to 1 ($\sum w_j = 1$), to ensure balanced contributions from all factors. Each weight reflects the importance of its respective criterion. For example, w_3 might be assigned a higher value in latency-sensitive applications where meeting deadlines is critical.

The first term, $\frac{1}{D_i}$, prioritizes tasks with shorter durations, as these can be completed quickly and improve overall throughput. By taking the reciprocal of D_i , tasks with smaller durations are assigned higher values, ensuring they receive higher priority. The second term, $\frac{1}{R_i}$, accounts for the computational resource requirements. Tasks requiring fewer resources are preferred, as they exert less load on fog nodes, allowing more tasks to be scheduled simultaneously.

Deadline proximity is captured in the third term, $\frac{1}{t_{\text{deadline},i}}$, where $t_{\text{deadline},i}$ time remaining until the task's deadline. Tasks closer to their deadlines are assigned higher priority, ensuring that they are executed promptly to avoid penalties. Dependency count, represented by $|D_i|$, is handled in the fourth term. Since tasks with more dependencies risk delaying dependent tasks if not executed on time, their priority is inversely proportional to $|D_i|$, with an offset of 1 to avoid division by zero. Lastly, the impact factor I_i directly contributes to the priority score. This factor is determined based on the task's criticality in the workflow, with higher values assigned to tasks that significantly influence system outcomes. To ensure consistency across different scales of these factors, normalization is applied. For example, duty cycles are normalized

TABLE 1: Existing work for Task Scheduling in Fog Computing

S.NO	Technique	Limitations	Simulation Tool	Parameters
[15]	Double blockchain-enabled task scheduling algorithms	Dependence on DBC maintenance higher energy overhead	MATLAB 2020a	Fog node count, malicious node computing power, transmission delay, bandwidth
[16]	Reinforcement learning (GA, SARSA, Q-learning)	potential scalability issues in real-world deployments	MATLAB	Throughput, latency, makespan, energy consumption
[17]	Bag-of-tasks scheduling using multi-objective optimization	Trade-offs between cost efficiency and system performance	Python	resource costs, task execution time
[18]	Improved Life-Choice-Based Optimization (ILCO) algorithm	scalability concerns for large-scale applications	Python	Power consumption, service latency, cost, computational resources
[19]	cyber-physical-social systems (CPSS)	Complexity in achieving optimal balance among multiple objectives	IFogSim	Task allocation, resource utilization, latency, system throughput
[20]	Mexican Axolotl Optimization (MAO) algorithm	Scalability issues for larger tasks	CloudSim	Makespan, response time, success rate, resource utilization, energy consumption
[21]	Deep Reinforcement Learning (DRL) based Virtual Network Embedding (VNE)	Computational overhead in dynamic scheduling	GT-ITM	Cost, VNR acceptance rate, computational resources, bandwidth, latency
[22]	Multi-Objective Simulated Annealing (MOSA) algorithm	Trade-off challenges between service delay and cost	MATLAB	Service delay, service cost, access level, task deadline
[23]	Star-Quake Multi-Objective Gravitational Search Algorithm (SQ-MOGSA)	Computational overhead in large-scale systems	MATLAB 2018b	Makespan, energy consumption, processing cost
[24]	Multi-agent Deep Reinforcement Learning (DRL)	Higher computational complexity for training; potential inefficiencies in real-time dynamic environments.	TensorFlow 2 with Python 3.7	Average utility, latency, energy consumption, queue waiting time
[25]	GWO, HEFT for resource allocation	Scalability issues for larger diverse systems reliance on HEFT parameters for task ordering.	IFogSim Toolkit	Makespan, throughput, task length, energy consumption, bandwidth
[26]	Energy-Efficient Internet of Medical Things (EEIoMT) framework	Limited scalability for large-scale IoMT	IFogSim2	Energy consumption, latency, network usage, CPU utilization
[27]	Convolutional Neural Network (CNN) combined with Modified Particle Swarm Optimization (MPSO)	Potential scalability challenges in large-scale fog environments	Python	Resource utilization, response time, makespan, load balancing level, total cost
[28]	Fog-enabled cloud framework (HealthFog) integrated with deep learning	Limited scalability in high-demand and resource allocation.	FogBus Framework	Latency, energy consumption, throughput, resource utilization
[29]	Reinforcement Learning with primary-backup task assignment for reliability in dynamic fog environments	Higher computational overhead during training	Python	Reliability, workload distribution, latency, throughput
[30]	Improved Ant Colony Optimization (ACO) with adaptive	Scalability concerns in larger Directed Acyclic Graph (DAG) models	MATLAB	Makespan, load balancing, energy consumption
[31]	OBCR (Opposition-Based Chemical Reaction Optimization) and modified CRO	Complexity in large-scale real-time systems potential overhead in high-heterogeneity environments.	IFogSim Toolkit	Makespan, stability, task execution time
[32]	Improved Artificial Hummingbird Algorithm integrated with Opposition-Based Learning, chaos mapping, and Differential Evolution for optimization	Complexity in large-scale dynamic fog systems computational	MATLAB R2018b	Makespan, cost, energy consumption, task length
[33]	Two-Stage Task Scheduling (2-ST) combining Naive Bayes and hybrid heuristic algorithms (ACO and PSO) for optimization	Limited scalability with larger IoT environments computational overhead from hybrid algorithms	IFogSim	Latency, energy consumption, network utilization, task response time, task priority
[34]	Hybrid GA-PSO algorithm combining Genetic Algorithm and Particle Swarm Optimization	Computational complexity, scalability issues	IFogSim Simulation	Execution time, response time, completion time, resource utilization, energy consumption
[35]	Multi-Objective Grey Wolf Optimizer (MGWO) with Pareto dominance for task scheduling optimization	Limited resource heterogeneity handling inefficiencies in extremely dynamic IoT environments.	MATLAB R2018b	Delay, energy consumption, task length, resource utilization
[36]	Multi-objective Harris Hawks Optimization (MoHHO) algorithm with Pareto-optimal front and adaptive task scheduling	Limited exploration in extremely dynamic environments	MATLAB 2020a	Delay, energy consumption, task arrival rate, resource utilization
[37]	Marine Predators Algorithm (MPA) integrated with polynomial mutation	Higher computational cost for dynamic environments limited scalability	MATLAB	Makespan, carbon emission rate, energy consumption, flow time

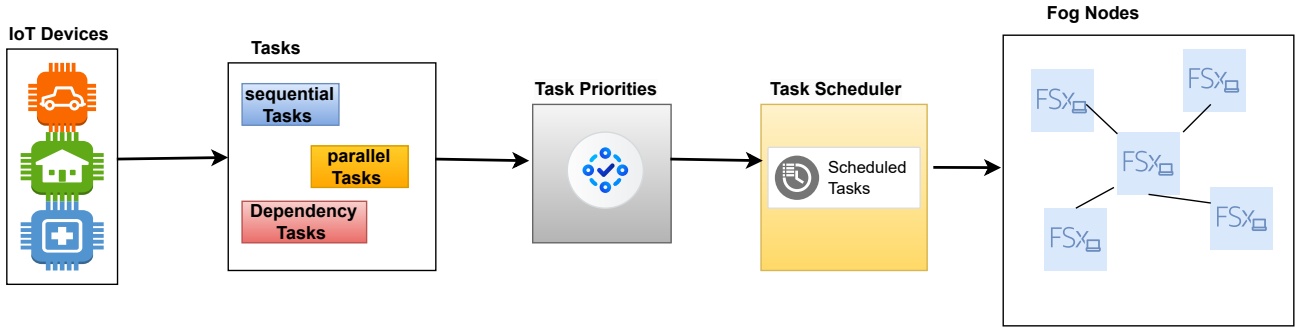


FIGURE 1: Proposed Methodology

$$D'_i = \frac{D_i}{\max(D)} \quad (2)$$

where $\max(D)$ is the maximum duration among all tasks. Similarly, resource requirements and deadlines are normalized:

$$R'_i = \frac{R_i}{\max(R)}, \quad t'_{\text{deadline},i} = \frac{t_{\text{current}}}{t_{\text{deadline},i}} \quad (3)$$

Normalization brings all criteria to a common level, allowing for fair comparison and preventing any single factor from disproportionately affecting the preference score. Substituting the normalized values into the general formula, the preference score becomes

$$P_i = w_1 \cdot \frac{1}{D'_i} + w_2 \cdot \frac{1}{R'_i} + w_3 \cdot \frac{1}{t'_{\text{deadline},i}} + w_4 \cdot \frac{1}{|D_i|} + w_5 \cdot I_i \quad (4)$$

Dynamic systems require adaptive priority strategies. For example, as conditions change during the execution of a task, such as its remaining duration or resource requirements, the priority score should be updated to reflect its current state. A dynamic update rule is defined as

$$P_i^{t+1} = P_i^t - \delta_t \cdot \left(\frac{\Delta D_i}{\Delta t} + \frac{\Delta R_i}{\Delta t} \right) \quad (5)$$

Here, δ_t is the adjustment rate and $\frac{\Delta D_i}{\Delta t}$ and $\frac{\Delta R_i}{\Delta t}$ represent the changes in time and resource requirements over time, respectively. This update ensures that tasks that are close to completion or require fewer resources are dynamically prioritized. In cases where multiple tasks have similar priority scores, tie-breaking mechanisms are used. For example, prioritizing tasks with the smallest $t_{\text{deadline},i}$, $\min(t_{\text{deadline},i})$ reduces the risk of missing critical deadlines. Prioritizing tasks that require less computational effort: $\min(R_i)$. Prioritizing tasks with fewer dependent tasks $\min(|D_i|)$.

1) Determination of Weights in Priority

The weights $\omega_1, \omega_2, \omega_3, \omega_4, \omega_5$ used in the priority calculation were determined through a systematic approach that combined theoretical deduction and empirical experiments.

Each weight reflects the relative importance of a task attribute in achieving optimal performance in a fog computing environment. Initially, theoretical deduction was used to assign values based on the difficulty of each attribute. For example, ω_1 (task duration) is moderately weighted to prioritize short tasks and reduce the overall make-span, ω_2 (resource requirements) is assigned low weighting to balance resource allocation without overloading fog nodes, ω_3 (deadline proximity) receives the highest weighting, emphasizing the importance of meeting task deadlines in latency-sensitive applications, ω_4 (dependency count) is lightly weighted to take into account the task importance in system performance, ensuring prompt execution of tasks with significant dependencies, preserving workflow integrity, and ω_5 (impact factor). These initial values were further refined through empirical experiments using multiple simulated task sets of different workloads in a fog computing environment. The simulations evaluated key performance metrics such as makespan, fault tolerance, energy efficiency, and scalability, which led to iterative adjustments in the weights. Notably, ω_3 was given the highest priority in deadline-critical cases, while ω_2 was tuned for resource-constrained setups. The final weight configuration $\omega_1 = 0.2, \omega_2 = 0.15, \omega_3 = 0.4, \omega_4 = 0.15, \omega_5 = 0.1$ ensures balanced prioritization of tasks, addressing system goals in accordance with dynamic conditions. The normalization of weights ensures fairness across different task characteristics, and the priority calculation mechanism dynamically adapts to real-time changes in workload characteristics.

B. SCHEDULING METHODOLOGY

The proposed framework simplifies task scheduling in the fog by organizing tasks into sequential, parallel, and dependency-based categories. It uses fog nodes equipped with multiple virtual machines (VMs) to ensure efficient execution while maintaining system scalability and reliability. Sequential tasks are executed in a strict order, where each task starts only after the previous one has completed. For example, tasks T_1, T_2 , and T_3 follow a clear linear workflow suitable for data pipelines. The scheduler dynamically assigns these tasks to the least-loaded VM, reducing waiting times and maintaining

Algorithm 1 CalculatePriority

Require: Task {duration, resources, deadline, impact, dependencies}
Require: Weights {w1, w2, w3, w4, w5}
Require: MaxValues {max_duration, max_resources, max_deadline}
Ensure: priority (calculated score)

```

1: Initialize variables:
2: if “duration”, “resources”, “deadline”, or “impact” are
   missing in Task then
3:   Print “Error: Task attributes are incomplete”
4:   Exit
5: end if
6: Normalize the task attributes:
7: duration_norm  $\leftarrow$  duration / max_duration
8: resources_norm  $\leftarrow$  resources / max_resources
9: deadline_norm  $\leftarrow$  deadline / max_deadline
10: Validate normalized values:
11: if duration_norm  $\leq 0$  or resources_norm  $\leq 0$  or deadline_norm  $\leq 0$  then
12:   Print “Error: Invalid normalization due to zero or
      negative input values”
13:   Exit
14: end if
15: Handle dependencies:
16: if “dependencies” is not in Task then
17:   dependencies  $\leftarrow 0$ 
18: else
19:   if dependencies  $< 0$  then
20:     Print “Error: Dependencies cannot be negative”
21:     Exit
22:   end if
23: end if
24: Compute priority factors:
25: priority  $\leftarrow 0$ 
26: for all { (“duration_norm”, w1), (“resources_norm”,
   w2), (“deadline_norm”, w3), (“impact”, w5) } do
27:   if factor = “impact” then
28:     priority  $\leftarrow$  priority + (weight  $\times$  impact)
29:   else
30:     priority  $\leftarrow$  priority + (weight / Task[factor])
31:   end if
32: end for
33: Include dependencies in priority:
34: if dependencies = 0 then
35:   priority  $\leftarrow$  priority + (w4 / 1)
36: else
37:   priority  $\leftarrow$  priority + (w4 / (dependencies + 1))
38: end if
39: Return the computed priority:
40: return priority

```

resource efficiency. Parallel tasks such as T_1 , T_2 , and T_3 are executed simultaneously on multiple VMs. This takes full advantage of the distributed nature of fog nodes, allowing for faster task completion by balancing loads and maximizing resource utilization. This is especially effective for independent tasks such as batch processing or data analysis.

For dependency-based tasks, execution is carefully controlled. Tasks such as T_6 wait until T_4 and T_5 before starting, ensuring logical consistency. The scheduler actively monitors dependencies and allocates tasks when prerequisites are met, which is suitable for workflows with complex tasks. Fog nodes act as the processing backbone, hosting multiple VMs to dynamically manage tasks. The scheduler monitors task distribution, adapting to real-time system conditions such as different resource loads or task priorities. By combining these strategies, the framework achieves a favorable balance between performance and energy efficiency, meeting the demands of modern IoT-based fog systems.

1) Sequential

Sequential task scheduling is one of the simplest but most complex approaches in fog computing, where tasks are executed in strict order one after the other. This strategy ensures that each task is completed before the next one begins, which is especially useful in cases where tasks are interdependent or where resources are limited to handle only one task at a time. Sequential scheduling is used to minimize overall latency, avoid resource idling, and complete tasks in a timely manner. In fog computing environments, each fog node is equipped with multiple virtual machines (VMs) to process tasks. Let $T = \{T_1, T_2, \dots, T_N\}$ denote the sequential execution times. Each task T_i is characterized by a task duration of D_i (the time required to execute). R_i is the resource requirement of the task. The priority score of the task is determined at the priority stage P_i .

The fog node is denoted as f_j , containing V VMs

$$f_j = \{VM_{j1}, VM_{j2}, \dots, VM_{jV}\}, \quad j = 1, 2, \dots, F \quad (6)$$

where VM_{jk} represents the k VM of the j fog node. The sequential scheduling process assigns each task T_i to the VM with the least current load, ensuring that resources are balanced across the available VMs. The VM assignment is expressed as

$$VM_{jk} = \arg \min_k L_{jk}, \quad \text{where } L_{jk} = \sum_{T_i \in VM_{jk}} R_i \quad (7)$$

Here, L_{jk} represents the current load on VM_{jk} , which is the sum of resource requirements R_i of all tasks currently assigned to the VM. The start time S_i task T_i count on the completion time C_{i-1} of the previous task in the sequence. The relationship is given as

$$S_i = \max(C_{i-1}, t_{\text{arrival},i}) \quad (8)$$

where $t_{\text{arrival},i}$ the time at which task T_i arrives at the fog node. The completion time C_i for T_i is

$$C_i = S_i + D_i \quad (9)$$

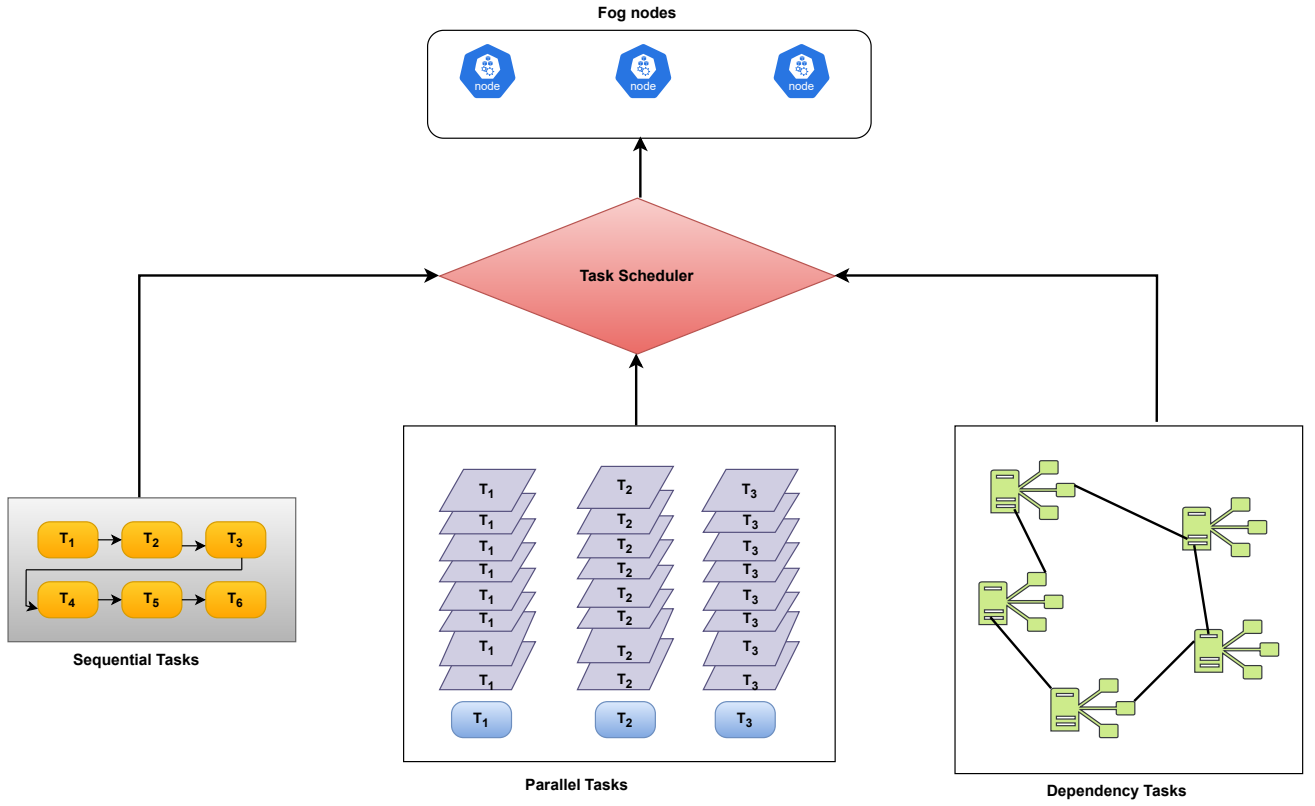


FIGURE 2: Task Scheduling

This ensures that tasks are executed strictly one after another, maintaining their sequential order. While tasks are executed sequentially, the assignment to VMs within the fog node ensures balanced resource utilization. The average load across all VMs in fog node f_j is calculated as

$$\text{Load}_{\text{avg}}(f_j) = \frac{1}{V} \sum_{k=1}^V L_{jk} \quad (10)$$

To maintain load balance, the load variance across VMs is minimized

$$\text{Load}_{\text{var}}(f_j) = \frac{1}{V} \sum_{k=1}^V (L_{jk} - \text{Load}_{\text{avg}}(f_j))^2 \quad (11)$$

Low variance implies that tasks are distributed evenly, preventing any single VM from becoming overloaded. In a dynamic fog environment, task arrival times are unpredictable, requiring tasks to be reassigned to different VMs based on real-time changes in resource availability. If a new task T_{new} arrives while T_i is being processed, the reassignment decision is based on minimizing the time to complete C_i .

$$C'_i = \min(C_i + D_{\text{new}}, t_{\text{current}} + D_{\text{new}}) \quad (12)$$

Here, C'_i is the updated completion time considering the reassignment of T_{new} .

Algorithm 2 Sequential

Require: List of tasks T , Fog nodes N

Ensure: Scheduled tasks with minimized makespan

- 1: **Initialize:** Sort tasks T by priority in descending order.
- 2: **for** each task $t \in T$ **do**
- 3: Find the least-loaded VM vm from N .
- 4: Assign t to vm .
- 5: Wait for t to complete.
- 6: Update the load of vm .
- 7: **end for**
- 8: **return** Updated task schedule and metrics.

2) Parallel

Parallel task scheduling is a strategy for executing multiple tasks simultaneously, which increases throughput and reduces overall makespan by efficiently utilizing available resources. In fog computing, parallel scheduling is particularly beneficial when there are independent tasks that do not depend on each other for execution. This approach ensures that all virtual machines (VMs) on a fog node are utilized efficiently, balancing the computational load and reducing task waiting times. Parallel scheduling refers to tasks that can be executed concurrently in a setup.

$$T_p = \{T_1, T_2, \dots, T_N\} \quad (13)$$

Each task T_i is defined by D_i The duration of the task. R_i The resource requirement for task execution. P_i The priority score of the task. Each fog node f_j contains V virtual machines (VM_{jk}) that can execute tasks concurrently. The assignment of tasks to VMs is based on their current load L_{jk} , which is the sum of the resource requirements of tasks already assigned to the VM. The goal is to allocate tasks to the n -least loaded VMs, where n is determined by the number of available VMs or tasks.

The assignment of task T_i was determined

$$A_p = \{VM_{jk_1}, VM_{jk_2}, \dots, VM_{jk_n}\}, \quad n = \min(|T_p|, V) \quad (14)$$

Here, A_p represents the set of VMs to which the parallel tasks are allocated, ensuring tasks are distributed across the least-loaded VMs. In parallel scheduling, each task T_i can start execution as soon as its assigned VM is available. The start time S_i is given by:

$$S_i = \max(t_{\text{arrival},i}, t_{\text{available},jk}) \quad (15)$$

where $t_{\text{arrival},i}$ is the time at which the task arrives fog node, and $t_{\text{available},jk}$ is the time when the assigned VM VM_{jk} becomes available. To ensure efficient utilization of resources, parallel scheduling incorporates load balancing by allocating tasks to the least-loaded VMs. The load on a VM VM_{jk} is updated as

$$L_{jk} \leftarrow L_{jk} + R_i \quad (16)$$

The average load across all VMs in the fog node is

$$\text{Load}_{\text{avg}}(f_j) = \frac{\sum_{k=1}^V L_{jk}}{V} \quad (17)$$

The objective is to minimize the load variance

$$\text{Load}_{\text{var}}(f_j) = \frac{1}{V} \sum_{k=1}^V (L_{jk} - \text{Load}_{\text{avg}}(f_j))^2 \quad (18)$$

In dynamic environments, new tasks T_{new} arriving during execution are allocated based on the updated load distribution. If A_p is the set of assigned VMs for existing tasks, the VM for the new task is chosen as:

$$VM_{jk} = \arg \min_k L_{jk} \quad (19)$$

The start time for the new task is

$$S_{\text{new}} = \max(t_{\text{arrival,new}}, t_{\text{available},jk}) \quad (20)$$

The updated load on VM_{jk} is recalculated to maintain balance.

3) Dependency

In fog computing, many tasks are interdependent, meaning that some tasks cannot begin execution until they are complete. These dependency-based tasks require careful management to ensure that the task execution order respects predefined task dependencies. Dependency-based scheduling

Algorithm 3 Parallel

Require: List of independent tasks T , Fog nodes N

Ensure: Concurrently executed tasks with optimized resource utilization

- 1: **Initialize:** Identify all independent tasks $T_{\text{indep}} \subseteq T$.
- 2: **for** each task $t \in T_{\text{indep}}$ **do**
- 3: Find an available VM vm from N .
- 4: Assign t to vm .
- 5: **end for**
- 6: Execute all tasks T_{indep} concurrently.
- 7: Update the load of each vm .
- 8: **return** Updated task schedule and metrics.

ensures that the task execution order adheres to these constraints, optimizing resource utilization. This type of scheduling is critical in workflows such as data processing pipelines, distributed systems, or task orchestration. In a dependency-based scheduling framework, tasks are processed only when all of their dependencies are complete.

$$T_d = \{T_1, T_2, \dots, T_N\} \quad (21)$$

The each task T_i represents among of tasks with dependencies associated. The task requires a duration of D_i to complete. R_i : Resource requirements to execute the task. D_i : The among of tasks that must be completed before T_i can start (i.e., its dependencies). P_i : The priority score of the task, determined in the priority stage. A fog node f_j , consisting of V VMs, schedules the tasks based on their readiness. A task T_i is ready to execute when all tasks in D_i have completed. The scheduling decision for each task is based on its readiness, and tasks are assigned to the least loaded VM

$$VM_{jk} = \arg \min_k L_{jk}, \quad \text{where } L_{jk} = \sum_{T_i \in VM_{jk}} R_i \quad (22)$$

Here, L_{jk} is the current load on VM_{jk} , which represents sum of the resource requirement R_i of all task assigned to that VM. For a task T_i , its readiness is determined by checking whether all tasks in D_i (its dependency set) have completed. The readiness function $R(T_i)$ is defined as

$$R(T_i) = \begin{cases} 1, & \text{all } T_d \in D_i \text{ completed} \\ 0, & \text{otherwise} \end{cases} \quad (23)$$

Once T_i is ready it is scheduled. The start time S_i for the ready task T_i is given by

$$S_i = \max(C_d \mid T_d \in D_i), \quad (24)$$

where C_d is the T_d completion time. The completion time C_i for task T_i is

$$C_i = S_i + D_i \quad (25)$$

This equation ensures that a task cannot be started until all its dependencies are completed, respecting the dependency constraints. The dependency structure affects the throughput

of the entire system. As tasks complete and their dependencies are resolved, the readiness of other tasks is updated. For each task T_i , after all tasks in D_i have completed, T_i is ready and is assigned to the next available VM. In dependency-based scheduling, tasks are assigned to the least loaded VM to ensure efficient resource utilization. The load on each VM is dynamically updated as tasks are assigned.

$$L_{j_k} \leftarrow L_{j_k} + R_i \quad (26)$$

This load balancing ensures that VMs are not overloaded and that resources are used efficiently across the entire fog node. As new tasks arrive in real time or as dependencies are resolved, tasks must be reassigned based on the current load of the VMs. If T_{new} arrives while other tasks are running, the allocation decision is made by selecting the VM with the least load.

where $t_{\text{arrival,new}}$ is the arrival time of the new task and $t_{\text{available,jk}}$ is the time when the selected VM becomes available for the new task.

Algorithm 4 Dependency

Require: List of tasks with dependencies T , Dependency graph G , Fog nodes N

Ensure: Tasks executed in dependency order

- 1: **Initialize:** Parse dependency graph G to determine task prerequisites.
 - 2: **while** there are unresolved tasks in T **do**
 - 3: Identify tasks $T_{\text{ready}} \subseteq T$ with all dependencies resolved in G .
 - 4: **for** each task $t \in T_{\text{ready}}$ **do**
 - 5: Find the least-loaded VM vm from N .
 - 6: Assign t to vm .
 - 7: Execute t on vm .
 - 8: Update dependency graph G to mark t as completed.
 - 9: **end for**
 - 10: **end while**
 - 11: **return** Updated task schedule with dependency constraints
-

C. METRICS

Makespan is one of the most important objectives in task scheduling, as it represents the total time required to complete all tasks. It is essential to improve throughput and ensure that tasks are completed as quickly as possible. In the context of fog computing, makespan is defined as the time it takes to complete all tasks on fog nodes and virtual machines (VMs). For a set of tasks $T = \{T_1, T_2, \dots, T_N\}$ with durations D_i assigned to VMs, makespan M is the maximum completion time of all tasks

$$M = \max(C_1, C_2, \dots, C_N) \quad (27)$$

where

$$C_i = S_i + D_i \quad (28)$$

C_i Completion time of task T_i . S_i Start time of task T_i . D_i Duration of task T_i . The completion time for task T_i depends on the previous task's completion time. In multiple tasks can be executed simultaneously, minimizing makespan, whereas in the makespan depends on task dependencies.

Fault tolerance refers to the ability of the system to continue functioning correctly even in the presence of faults or failures. In fog computing, this means that tasks should be executed even if some fog nodes or VMs fail. Fault tolerance ensures that tasks are not lost and that resources are reallocated dynamically to ensure continuity of service. Suppose the task completion time in the absence of faults is C_i , and the system can handle a certain number of failures f . The adjusted completion time C'_i after failure recovery can be defined as

$$C'_i = C_i + f \cdot \Delta \quad (29)$$

where f Number of faults. Δ Additional time needed to recover from each fault. Fault tolerance can be improved by replicating tasks across multiple VMs. In *sequential scheduling*, fault tolerance can be achieved by reassigning tasks to different VMs if a failure occurs. Requires ensuring that dependent tasks are rerouted if any of the prerequisite tasks fail.

Scalability is the ability of the system to handle an increasing number of tasks and resources efficiently. In fog computing, scalability is vital because the number of IoT devices and tasks can grow dynamically, and the fog computing system must scale to accommodate this increase without a drastic drop in performance. Scalability is measured by how well the system performance (e.g., makespan) increases as the number of tasks N or resources R increases. Let $M(N)$ be the makespan for N tasks, and $M(N + \Delta N)$ be the makespan after adding ΔN tasks

$$\text{Scalability} = \frac{M(N + \Delta N) - M(N)}{\Delta N} \quad (30)$$

The ideal scalability metric is one where the increase in makespan is minimal as N increases. Trust ensures that tasks are executed in a secure and reliable environment, and that the nodes (including VMs) behave as expected. The trust score τ_j of node f_j is quantified based on its historical performance:

$$S_j = \frac{\text{Number of tasks successfully executed on } f_j}{\text{Total number of tasks assigned to } f_j} \quad (31)$$

$$\tau_j = \alpha \cdot S_j + \beta \cdot \text{Security}_j \quad (32)$$

where α and β are weights, and Security_j represents the node's security level. Maximize the trust score τ_j .

Reliability measures the ability of a fog computing system to consistently perform its tasks over time without failure. The reliability of a fog node f_j is R_j .

$$R_j = \frac{\text{Number of tasks completed without failure}}{\text{total number of tasks assigned to } f_j} \quad (33)$$

The overall reliability of the system R_{sys}

$$R_{sys} = \prod_{j=1}^F R_j \quad (34)$$

Where F is the number of fog. In fog computing, energy consumption is a critical issue. Reducing energy consumption without sacrificing performance is key. Energy consumption E_i for task T_i

$$E_i = \epsilon \cdot R_i \cdot D_i \quad (35)$$

Where ϵ was energy efficiency of the fog node. R_i is the resource requirement of the task. D_i is the duration of the task.

TABLE 2: Summary of Notations and Their Descriptions

Notation	Description
T	All tasks
N	Total tasks
F	Total fog nodes
V	Number of VM per fog node
T_i	Task i
D_i	Duration required to complete task i
R_i	Resource requirements for executing task i
C_i	Completed time for task i
S_i	Start time for task i
L_i	Load or demand of task i
VM_{jk}	k -th virtual machine of the j -th fog node
W_{jk}	Workload of k -th VM of j -th fog node
C_{jk}	CPU capacity k -th VM of j -th fog node
M	Makespan
ϵ	Energy efficiency parameter
δ_t	Adjustment rate for updating priority scores and time step delta
ΔD_i	Change in duration of task i over time
ΔR_i	Change in resource requirement of task i over time
s	State
a	Action
r	Reward
γ	Discount factor
$\mu(s \theta^\mu)$	Policy function
$Q(s, a \theta^Q)$	Value function
$N(0, \sigma)$	Noise process added to action selection for exploration
τ	Soft update rate parameter
θ^μ, θ^Q	value networks
$\theta^{\mu'}, \theta^{Q'}$	Parameters of the target networks
L	Loss function used to update network parameters
J	Objective function

IV. PROPOSED TASK SCHEDULING FRAMEWORK OF DDPG

Task scheduling in the context of fog computing is a challenging problem that deals with mapping computational tasks to resources such as virtual machines while optimizing a set of key performance metrics. Metrics considered include makespan (total time to complete all tasks), energy consumption minimization, and trust and reliability maximization in the scheduling process. Fog computing brings computational resources closer to end devices, reducing latency and improving system performance. However, this proximity also introduces challenges such as the heterogeneity of tasks, the

dynamic nature of resources, and the need to manage conflicting goals. Traditional scheduling methods, which often rely on fixed rules or heuristics, struggle to keep up with these complexities. In response, reinforcement learning methods such as DDPG have emerged as powerful tools for optimizing scheduling decisions in such environments.

DDPG is a reinforcement learning algorithm specifically designed for problems with continuous or high-dimensional discrete action spaces. It combines the strengths of decision-making policy gradients (DPG) that allow for decision-making action selection, and deep Q-networks (DQN), which use neural networks to estimate a Q-value function. This makes DDPG a particularly suitable task scheduling tool in fog computing, where the assignment of work to specific VMs involves an action space. Furthermore, DDPG uses an actor-critic architecture that allows for efficient policy learning. The actor network learns a decision-making policy that maps actions, while the critic network evaluates the quality of these actions using a Q-value function. This architecture allows DDPG to dynamically optimize scheduling strategies that interact with the environment and learn from the feed.

The task scheduling problem in fog computing could be formalized as MDP. The MDP is defined by (S, A, P, R, γ) , where S is state space, A is action space, P represents the state transition probabilities, R was the reward function, and $\gamma \in [0, 1]$ is a discount factor balances the immediate and future rewards. The state space S represents the current configuration of the environment, capturing both the characteristics of the task to be scheduled and the state of the available computational resources. For a given task t_i , task-specific characteristics include task length (L_i), resource requirement (R_i), deadline (D_i), and impact factor (I_i). Together, these attributes form the task feature vector $[L_i, R_i, D_i, I_i]$. On the resource side, the state captures the state of the VMs, such as their current workload ($W_{j,k}$) and CPU capacity ($C_{j,k}$) for all M fog nodes and their N VMs. The parameter metrics such as current makespan ($T_{makespan}$, energy consumption (E), system trust (T_{trust}) are also included to provide. The full state at t is denoted as

$$s_t = [L_i, R_i, D_i, I_i, W_{1,1}, C_{1,1}, \dots, W_{M,N}, C_{M,N}] \quad (36)$$

The action space A in task scheduling consists of decisions to assign tasks to specific VMs. If there are M fog nodes and N VMs per node, the action space includes $M \times N$ discrete actions, each representing the selection of a specific VM for task execution. The actor network in DDPG outputs a policy $\mu(s_t | \theta^\mu)$, which maps states s_t , actions a_t . During training, exploratory noise $N(0, \sigma)$ is added to the output of the actor network to encourage exploration

$$a_t = \mu(s_t | \theta^\mu) + N(0, \sigma) \quad (37)$$

The reward function R_t is crucial in guiding the learning process and ensuring the agent optimizes the desired objectives. In fog task scheduling, the reward function incorporates

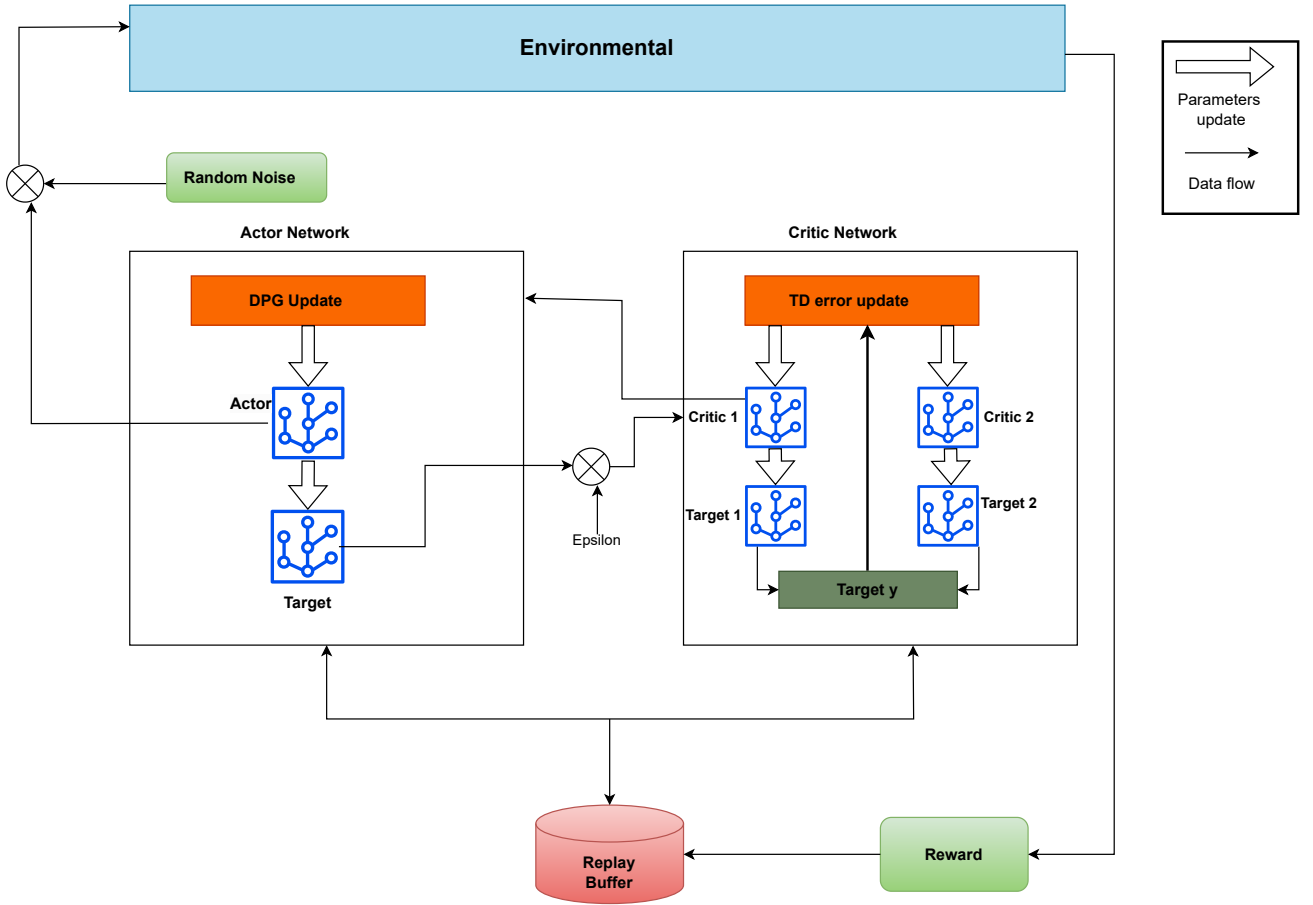


FIGURE 3: Task Scheduling using Deep Deterministic Policy Gradient(DDPG)

multiple objective such as reducing the makespan, reducing energy consumption, and maximize the trust. It is defined as

$$R_t = -\frac{T_{\text{makespan}}}{T_{\text{total}}} - \alpha \frac{E}{E_{\text{max}}} + \beta T_{\text{trust}} + \gamma P_{\text{priority}} \quad (38)$$

where T_{total} is the total execution time of all tasks, E_{max} is the maximum allowable energy consumption, and α, β, γ are coefficients that balance the importance of the different objectives. The priority score P_{priority} for each task is computed as

$$P_{\text{priority}} = \omega_1 L_i + \omega_2 R_i + \omega_3 D_i + \omega_4 I_i \quad (39)$$

where $\omega_1, \omega_2, \omega_3, \omega_4$ are weights representing the relative importance of task attributes. DDPG uses two neural networks: the actor network, which learns the policy $\mu(s | \theta^\mu)$ outputting optimal actions, and the critic network, which estimates the Q-value $Q(s, a | \theta^Q)$. The Q-value expresses the expected cumulative reward that one can get by taking the action a in state s . The Q-value updates according to the Bellman equation:

$$Q(s_t, a_t | \theta^Q) = r_t + \gamma Q(s_{t+1}, a_{t+1} | \theta^Q) \quad (40)$$

where γ is the discount factor. The critic is train to minimize the loss

$$L_{\text{critic}} = \mathbb{E}[(y_t - Q(s_t, a_t | \theta^Q))^2] \quad (41)$$

where y_t is the target Q-value

$$y_t = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1} | \theta^{\mu'}) | \theta^Q) \quad (42)$$

The actor network is updated using the policy gradient:

$$\nabla_{\theta^\mu} J = \mathbb{E}[\nabla_a Q(s, a | \theta^Q) |_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s | \theta^\mu)] \quad (43)$$

Target networks are used for both the actor and critic to stabilize training. These networks are slowly updated towards the weights of their respective primary networks using a soft update rule

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (44)$$

where $\tau \in (0, 1)$ controls the update rate. The training process starts by initializing a replay buffer to hold the actor and critic networks, target networks, and previous experiences. At each time step, an agent observes the current state s_t and chooses action a_t using the actor network policy, adding exploratory noise. This action is taken in the environment, resulting in a new state s_{t+1} , a reward r_t , and an end flag. This transition (s_t, a_t, r_t, s_{t+1}) is added to the replay buffer.

In each iteration, a batch of transitions from the buffer is sampled to update the networks. The critic is updated by minimizing the loss L_{critic} , and the actor is updated using the policy gradient. The soft update rule updates the target networks. DDPG is suitable for the fog computing environment due to its ability to handle large action spaces and optimize multiple objectives in dynamic environments. By learning through interactions with the environment, the algorithm efficiently allocates tasks to resources, leading to lower makepan, energy consumption, and improved reliability. These capabilities position DDPG as a robust and scalable solution for modern fog computing systems.

V. EXPERIMENTAL RESULTS

This section shows the experimental results obtained from the simulation of the proposed DDPG-based scheduling framework. The simulation setup datasets and evaluation metrics are described after the framework performance analysis when compared to baseline algorithms. Simulation environment to test the proposed DDPG-based framework under different workload conditions. The jobs used from the Google Cloud Jobs [44] dataset, which was originally designed for cloud-based job scheduling and resource management, are well suited for integration into the fog computing environment described in this study. This dataset provides important features such as job duration, resource requirements, dependencies, and priority levels, which are very suitable for simulating real-world workload scenarios in fog computing. By adjusting for fog computing constraints such as decentralized resources and improved data privacy, the dataset ensures that algorithms can be rigorously tested and optimized for efficiency, scalability, and responsiveness in distributed computing environments. Its integration provides a solid foundation for evaluating and advancing task scheduling strategies that address the challenges of fog computing, thereby improving the reliability and performance of IoT-based applications, which are categorized into three groups: small, medium, and large based on task complexity and computational requirements. Each dataset represents a range of task lengths and priority levels to simulate real-world fog [14].

The simulated fog nodes represent an environment with different capacities, from CPU and memory capacity to bandwidth. The generalized properties of the tasks include task duration, task dependencies, and resource usage, which are processed before scheduling. All simulation tasks were performed using Simpy, a Python-based tool. The experimental evaluation covered multiple key aspects of task scheduling on fog nodes. The proposed framework was able to dynamically assign tasks to virtual machines and optimize resource usage while minimizing overhead. The results also highlighted the adaptability of the DDPG framework to changing task demands and resource availability. It benchmarks its performance against established algorithms and compares the improvements in scheduling efficiency. Some of the important comparisons include task completion rates, fault tolerance, and resource utilization for small, medium,

Algorithm 5 Deep Deterministic Policy Gradient

```

1: Actor  $\mu(s)$  with random weights
2: Critic  $Q(s, a)$  with random weights
3: Target  $\mu'(s) \leftarrow \mu(s)$  and  $Q'(s, a) \leftarrow Q(s, a)$ 
4: Replay buffer  $R$  with size max_capacity
5: Set hyper parameters:  $\gamma$  (discount factor),  $\tau$  (target update rate),
                                      $\alpha_{\text{actor}}, \alpha_{\text{critic}}$  (learning
                                     rates)
6: for episodes = 1 to max_episodes do
7:   Reset environment and observe initial state  $s_0$ 
8:    $total\_reward \leftarrow 0$ 
9:    $done \leftarrow \text{False}$ 
10:  while not  $done$  do
11:    Action Selection:
12:    if random() < exploration_probability
13:      then
14:         $a \leftarrow$  Random action from action space
15:      else
16:         $a \leftarrow \mu(s) + \text{Noise for exploration}$ 
17:      end if
18:    Execute action  $a$  and observe  $s', r$ , and  $done$ 
19:    Store transition  $(s, a, r, s', done)$  in  $R$ 
20:     $total\_reward \leftarrow total\_reward + r$ 
21:    if len(R) ≥ batch_size then
22:      Sample minibatch  $(s_i, a_i, r_i, s'_i, done_i)$ 
23:      from  $R$ 
24:      Critic Update:
25:      Compute target Q-values:
26:       $y_i \leftarrow r_i + \gamma(1 - done_i)Q'(s'_i, \mu'(s'_i))$ 
27:      Compute critic loss:
28:       $L_C \leftarrow \frac{1}{N} \sum_i (y_i - Q(s_i, a_i))^2$ 
29:      Minimize  $L_C$  to update Critic parameters
30:      Actor Update:
31:      Compute actor loss:
32:       $L_A \leftarrow -\frac{1}{N} \sum_i Q(s_i, \mu(s_i))$ 
33:      Minimize  $L_A$  to update Actor parameters
34:      Soft Update Target Networks:
35:      for each parameter pair  $(\theta_{\text{target}}, \theta_{\text{source}})$  do
36:         $\theta_{\text{target}} \leftarrow \tau\theta_{\text{source}} + (1 - \tau)\theta_{\text{target}}$ 
37:      end for
38:    end if
39:     $s \leftarrow s'$ 
40:  end while
41:  Store  $total\_reward$  for analysis (optional)
42: end for

```

and large datasets. These analyses showed the superiority of the DDPG framework for handling high-priority tasks with complex dependencies and minimal latency and energy consumption.

This study explores the complex trade-offs inherent in fog computing task scheduling, focusing on key performance indicators such as makespan, fault tolerance, energy efficiency, and scalability. The Deep Deterministic Policy Gradient (DDPG) algorithm serves as the cornerstone of our approach, dynamically balancing these metrics to optimize system performance. DDPG operates in a continuous action space, allowing for precise adjustments in task assignments based on real-time conditions and task priorities. Its actor-critical architecture enables simultaneous learning of optimal policies and evaluation of their effectiveness, ensuring that tasks are prioritized to minimize makespan while maintaining strong fault tolerance and efficient resource utilization. By continuously updating its policies through reinforcement learning, DDPG adapts to varying workload demands and environmental changes, achieving a harmonious balance between competing performance goals. This adaptive capability not only increases the reliability and scalability of fog computing systems but also supports sustainable energy practices by optimizing resource utilization over time. The proposed work is compared with the baseline algorithms DQN, A2C, and A3C [13] [14].

TABLE 3: Simulation Parameters

Simulation Parameters	Details
Number of Tasks	100–1000 tasks
Task Lengths	15,000–900,000 computational units
Number of Fog Nodes	10
Virtual Machines per Node	6
Bandwidth	200 MBPS (per VM)
Operating System	Linux
Processor	Intel i7
Simulation tool	SimPy

A. MAKESPAN

MakeSpan is a key metric in scheduling that measures the total time required to complete all tasks. It is crucial for improving resource utilization, reducing task completion time, and improving system performance on fog nodes. Effective management of MakeSpan ensures balanced workload distribution across fog nodes, which leads to better responsiveness and optimal resource utilization. In this study, MakeSpan was evaluated using the experimental settings described in Table 3. The workloads were taken from realistic datasets and categorized into three groups: small (datasets of 100-350 tasks), medium (datasets of 400-650 tasks), and large (datasets of 700-1000 tasks). The tasks that simulate different types of fog computing tasks range from 10,000 to 800,000 computational units. The results for small datasets are presented in Table 4 of the DDPG algorithm's performance, which is compared to baseline methods such as DQN, A2C, and A3C. For tasks ranging from 100 to 350, DDPG consistently performed better, with a 25-35% reduction in makespan compared to

DQN and a 15-20% improvement over A3C, as shown in Figure 4. The DDPG algorithm continues to outperform the baseline approaches for medium datasets, as illustrated in Table 5, achieving a 28% reduction in makespan compared to A2C and an 18% improvement over A3C for workloads between 400 and 650 tasks. These results are depicted in Fig. 5. For large datasets (700-1000 tasks), DDPG demonstrated excellent scalability, as summarized in Table 6. It reduced the makespan by up to 30% compared to the next best performing algorithm, A3C. A diagram representing the results can be seen in Figure 6.

TABLE 4: Makespan for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	992.32	826.93	719.07	653.7
Dataset150	1138.34	948.62	824.89	749.9
Dataset200	1249.62	1041.35	905.52	823.2
Dataset250	1357.85	1131.54	983.95	894.5
Dataset300	1464.42	1220.35	1061.17	964.7
Dataset350	1605.89	1338.24	1163.69	1057.9

TABLE 5: Makespan for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	1706.53	1422.11	1236.62	1124.2
Dataset450	1905.24	1587.7	1380.61	1255.1
Dataset500	1993.13	1660.94	1444.3	1313.0
Dataset550	2119.74	1766.45	1536.04	1396.4
Dataset600	2221.9	1851.58	1610.07	1463.7
Dataset650	2336.65	1947.21	1693.23	1539.3

TABLE 6: Makespan for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	2456.74	2047.28	1780.24	1618.4
Dataset750	2571.19	2142.66	1863.18	1693.8
Dataset800	2682.46	2235.38	1943.81	1767.1
Dataset850	2777.03	2314.19	2012.34	1829.4
Dataset900	2897.26	2414.38	2099.46	1908.6
Dataset950	3011.26	2509.38	2182.07	1983.7
Dataset1000	3145.14	2620.95	2279.09	2071.9

B. TRUST

Trust is a key metric in scheduling, especially in fog computing nodes, where the reliability and security of fog nodes significantly affect the overall system performance. A high trust score ensures that computational tasks are executed efficiently and securely, which reduces the risk of outages caused by unreliable nodes. In this study, the trust metric was evaluated using the experimental setup described in Table 3. The reliability of the fog is a function of the successful completion of tasks and the security levels of the nodes. The datasets used were classified into three groups: small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks). For small datasets, DDPG demonstrated superior performance compared to baseline algorithms such as DQN, A2C, and A3C, as shown in Table 7. DDPG's trust score

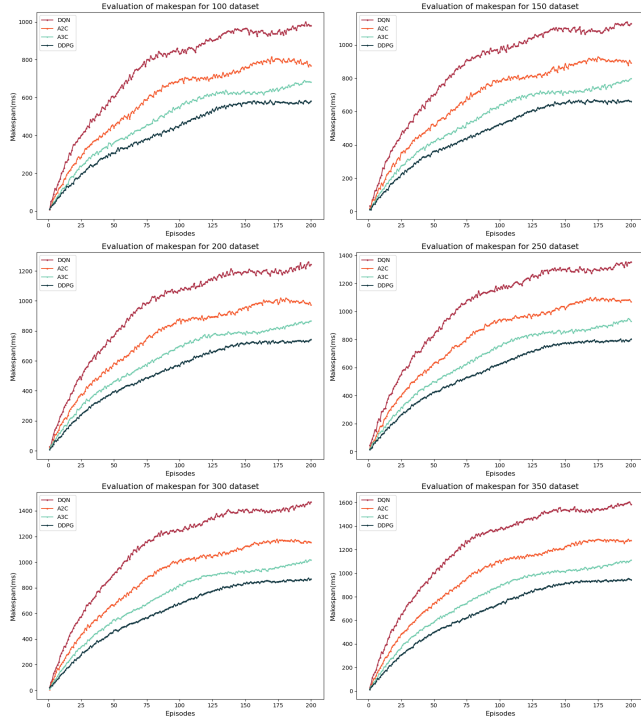


FIGURE 4: Makespan of 100_350 datasets

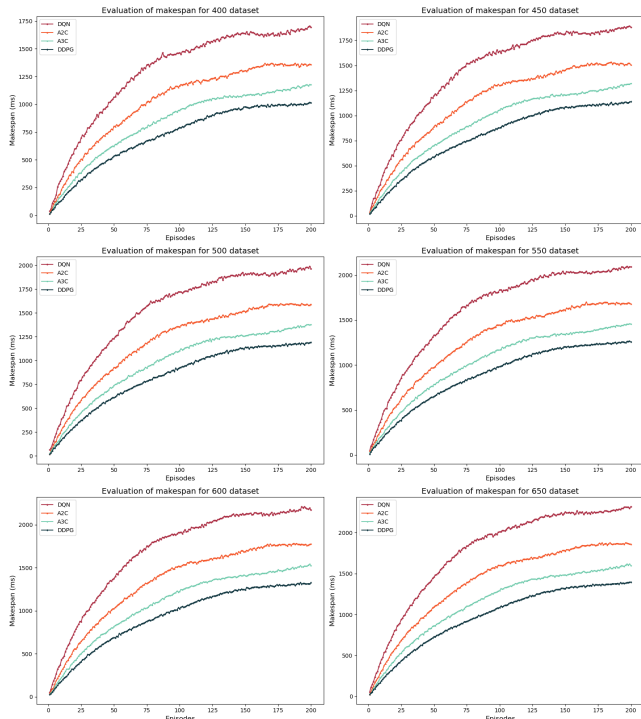


FIGURE 5: Makespan of 400_650 datasets

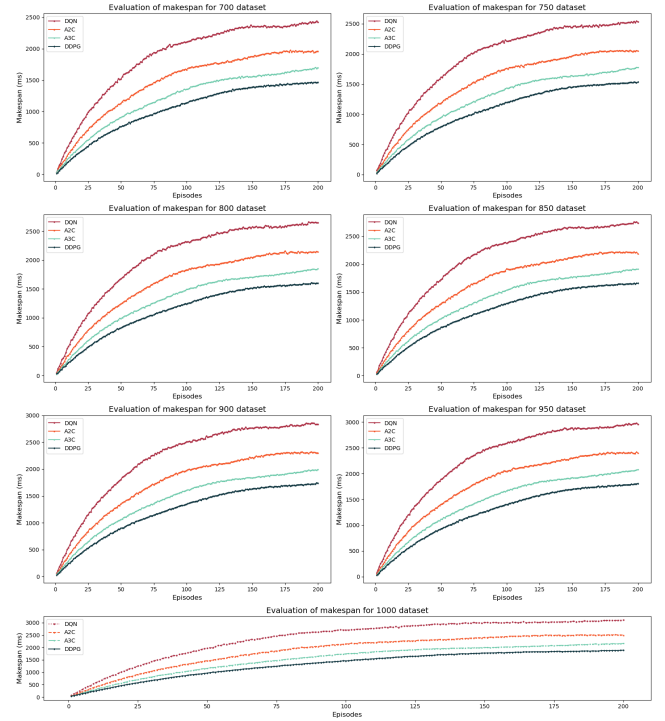


FIGURE 6: Makespan of 700_1000 datasets

ranged from 0.91 to 0.94, while A3C and A2C achieved scores between 0.75 and 0.80, while DQN scored from 0.55 to 0.60. These results are visually represented in Figure 7. When evaluating medium datasets, DDPG continues to outperform other algorithms as illustrated in Table 8. It achieved trust scores ranging from 0.93 to 0.96, higher than A3C (0.78 to 0.89) and A2C (0.65 to 0.78). These results, shown in Figure 8, highlight the robustness of DDPG in maintaining trust even as task complexity increases. For large datasets (700-1000 tasks), as summarized in Table 9, DDPG continued its leading performance, achieving trust scores ranging from 0.97 to 0.99. This marked a significant improvement over A3C, which scored between 0.88 and 0.90, and A2C, which scored between 0.75 and 0.85. The results are shown in Figure 9. The results show that the DDPG algorithm effectively increases trust in fog computing environments by executing secure and reliable tasks. Its ability to dynamically adapt to the environment and prioritize trusted nodes ensures high task completion rates and minimal security risks. This positions DDPG as a robust solution for trust-aware scheduling in dynamic and distributed fog computing systems.

TABLE 7: Trust for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	0.575	0.684	0.798	0.917
Dataset150	0.544	0.687	0.764	0.933
Dataset200	0.519	0.684	0.784	0.942
Dataset250	0.562	0.648	0.749	0.932
Dataset300	0.557	0.698	0.786	0.921
Dataset350	0.584	0.686	0.765	0.906

TABLE 8: Trust for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	0.574	0.648	0.787	0.958
Dataset450	0.541	0.696	0.757	0.932
Dataset500	0.565	0.684	0.765	0.929
Dataset550	0.545	0.684	0.795	0.955
Dataset600	0.565	0.684	0.746	0.964
Dataset650	0.584	0.629	0.768	0.987

TABLE 9: Trust for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	0.546	0.689	0.715	0.975
Dataset750	0.587	0.665	0.765	0.934
Dataset800	0.546	0.659	0.754	0.989
Dataset850	0.554	0.679	0.758	0.965
Dataset900	0.565	0.698	0.765	0.943
Dataset950	0.546	0.656	0.765	0.970
Dataset1000	0.564	0.685	0.787	0.966

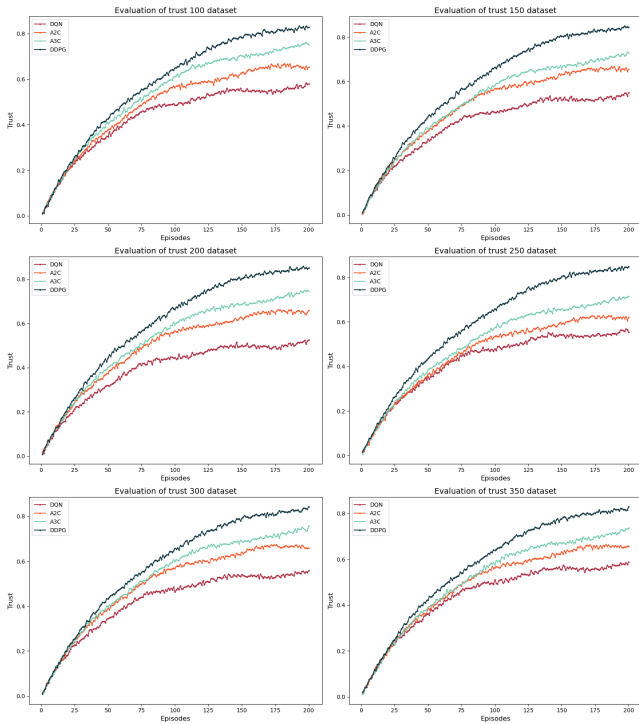


FIGURE 7: Trust of 100_350 datasets

C. SCALABILITY

Scalability is a key metric in fog computing, as it measures the ability of a system to handle increasing workloads and resource demands without significant degradation in performance. An efficient task scheduling algorithm ensures scalability, allowing the system to adapt to varying task loads while maintaining efficiency and responsiveness. In this study, scalability was evaluated using the experimental settings described in Table 3. The evaluation included datasets categorized as small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), which simulate the

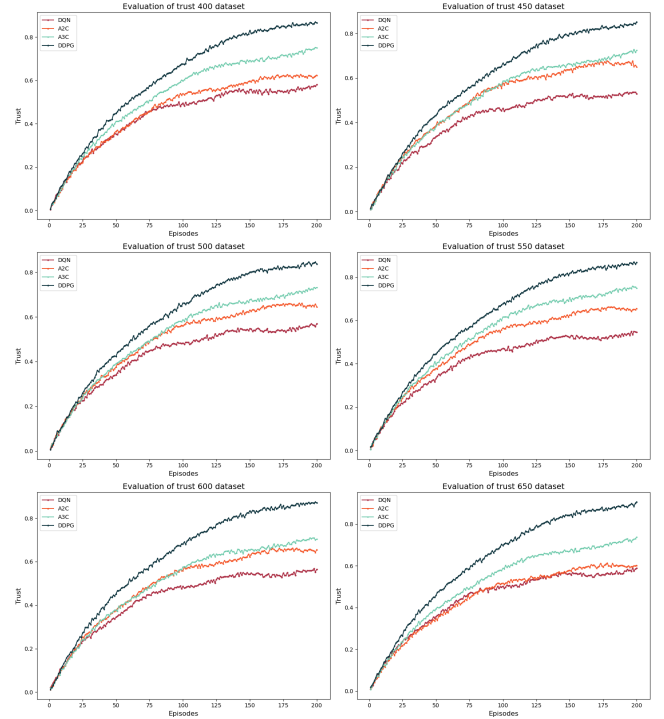


FIGURE 8: Trust of 400_650 datasets

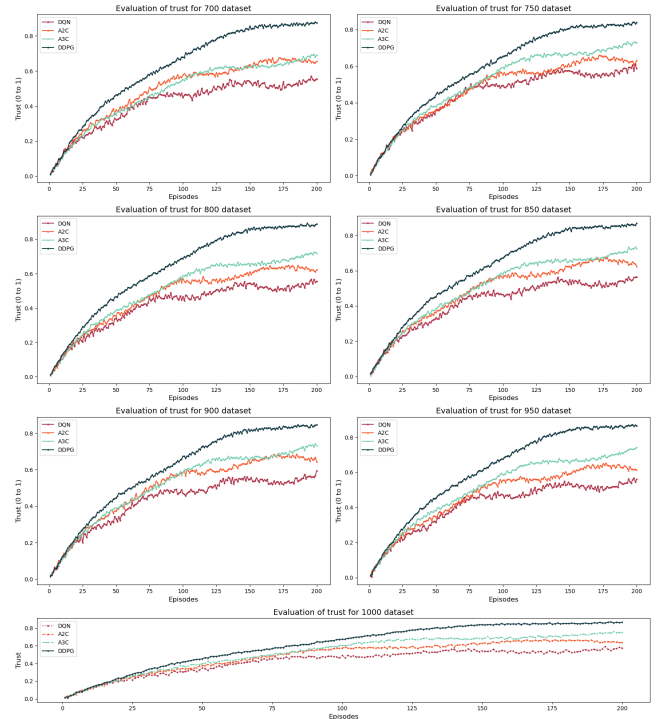


FIGURE 9: Trust of 700_1000 datasets

increasing complexity and size of tasks typically encountered in fog computing environments. For small datasets, DDPG outperformed the baseline algorithms including DQN, A2C, and A3C, as shown in Table 10. The scalability values for DDPG ranged from 0.61 to 0.79, which was significantly higher than A3C (0.55 to 0.67) and A2C (0.45 to 0.56). These results are visually depicted in Figure 10. For medium datasets, DDPG continued its superior performance, as shown in Table 11. The scalability values ranged from 0.72 to 0.88, which was significantly higher than A3C (0.58 to 0.77) and A2C (0.47 to 0.65). The scalability improvement is clearly visible from the result shown in Figure 11. In the case of large datasets (700-1000 tasks), as summarized in Table 12, DDPG demonstrated excellent scalability with values ranging from 0.87 to 0.98. This marked a significant improvement over A3C, which achieved 0.69 to 0.86, and A2C, which scored between 0.59 and 0.74. These trends are visually represented in Figure 12. The results confirm that DDPG effectively supports scalability in fog computing environments. By dynamically balancing workloads and efficiently utilizing available resources, DDPG ensures that system performance remains robust even as workload volumes and resource demands increase. This compatibility makes it a strong candidate for scheduling in scalable and distributed fog computing systems.

TABLE 10: Scalability for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	0.467	0.454	0.554	0.613
Dataset150	0.483	0.489	0.582	0.697
Dataset200	0.468	0.471	0.576	0.647
Dataset250	0.496	0.563	0.667	0.791
Dataset300	0.551	0.495	0.591	0.758
Dataset350	0.448	0.485	0.587	0.744

TABLE 11: Scalability for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	0.418	0.477	0.578	0.721
Dataset450	0.526	0.515	0.618	0.793
Dataset500	0.489	0.554	0.655	0.838
Dataset550	0.565	0.572	0.672	0.885
Dataset600	0.555	0.565	0.664	0.842
Dataset650	0.511	0.588	0.689	0.867

TABLE 12: Scalability for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	0.454	0.597	0.696	0.878
Dataset750	0.515	0.614	0.711	0.889
Dataset800	0.554	0.676	0.774	0.966
Dataset850	0.678	0.691	0.794	0.977
Dataset900	0.655	0.714	0.818	0.983
Dataset950	0.673	0.743	0.865	0.972
Dataset1000	0.646	0.728	0.825	0.986

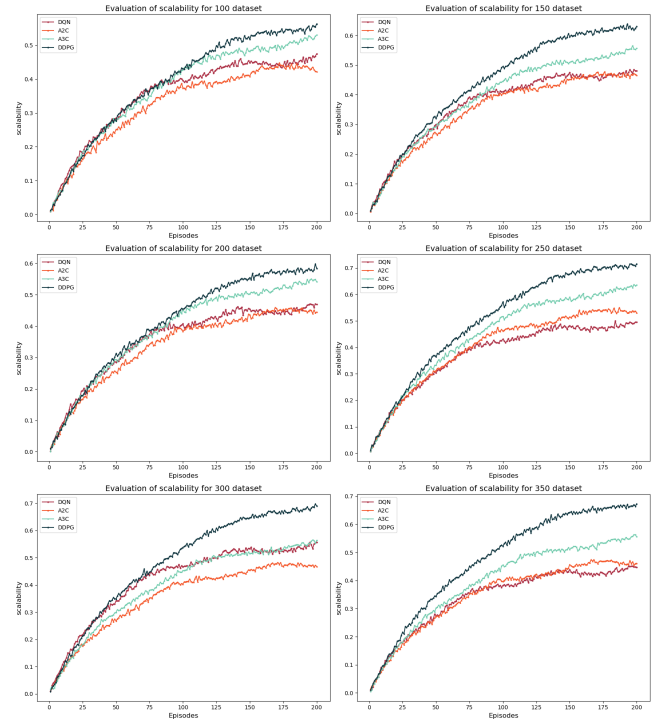


FIGURE 10: Scalability of 100_350 datasets

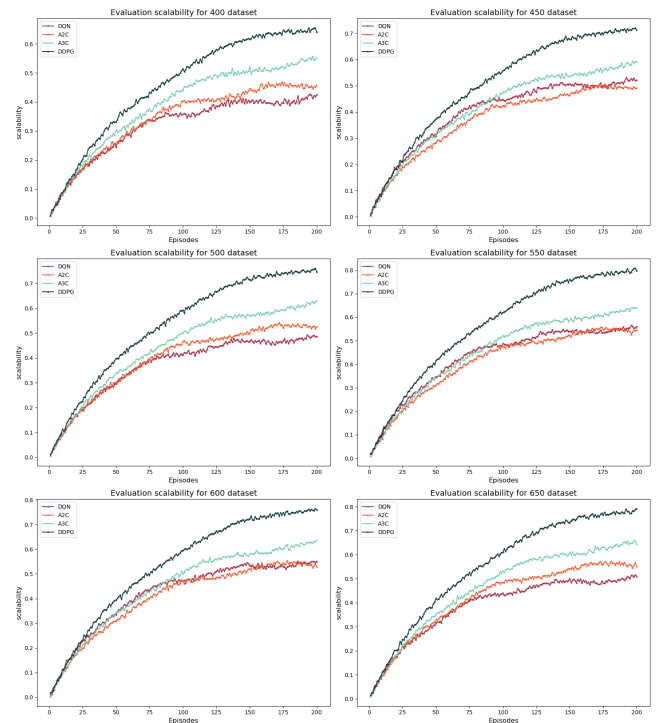


FIGURE 11: Scalability of 400_650 datasets

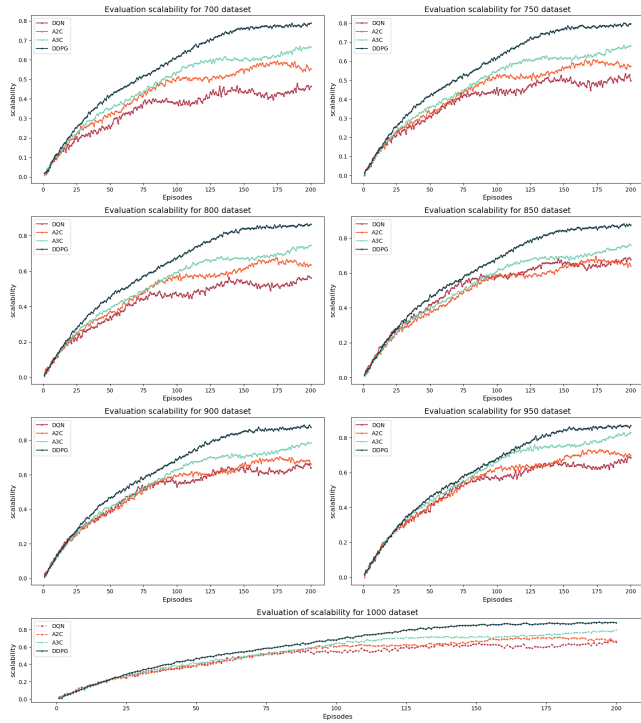


FIGURE 12: Scalability of 700_1000 datasets

D. ENERGY CONSUMPTION

Energy consumption is a very important metric in fog computing. Its impact is directly on both the operational efficiency and stability of the system. A good task scheduler reduces energy consumption but never degrades performance, makes nodes last longer, and reduces maintenance costs. This study evaluated energy consumption using the experimental setup described in Table 3. Workloads were divided into three categories: small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks). Each dataset was designed to reflect real-world energy demands in different fog computing scenarios. The results in Table 13 show that for small datasets, DDPG achieved significantly lower energy consumption compared to baseline methods such as DQN, A2C, and A3C. The energy consumption of DDPG ranged from 46.2 to 227.4 units, compared to 50.8 to 250.1 units for A3C and 58.4 to 287.6 units for A2C. These results are illustrated in Figure 13. On medium datasets, DDPG continues to outperform its counterparts, as illustrated in Table 14. The energy consumption for DDPG ranged from 264.9 to 452.7 units, a clear improvement over A3C (291.4 to 497.9 units) and A2C (335.1 to 572.6 units). The trends are visualized in Figure 14. For large datasets, DDPG demonstrated exceptional energy efficiency, as shown in Table 15. It achieved energy consumption values ranging from 483.1 to 764.7 units, outperforming A3C (531.4 to 841.2 units) and A2C (611.1 to 967.4 units). These results are shown in Fig15. The experimental results highlight the ability of DDPG to significantly reduce energy consumption compared

to other algorithms. By dynamically allocating resources and efficiently managing workloads, DDPG ensures minimal energy consumption without compromising performance. This makes DDPG a highly effective solution for energy-aware scheduling in fog computing environments.

TABLE 13: Energy Consumption for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	70.13	58.44	50.82	46.2
Dataset150	136.01	113.34	98.56	89.6
Dataset200	216.92	180.77	157.19	142.9
Dataset250	249.56	207.97	180.84	164.4
Dataset300	294.04	245.03	213.07	193.7
Dataset350	345.19	287.66	250.14	227.4

TABLE 14: Energy Consumption for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	402.12	335.1	291.39	264.9
Dataset450	453.12	377.6	328.35	298.5
Dataset500	487.73	406.44	353.43	321.3
Dataset550	552.85	460.71	400.62	364.2
Dataset600	591.26	492.72	428.45	389.5
Dataset650	687.2	572.67	497.97	452.7

TABLE 15: Energy Consumption for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	733.34	611.12	531.41	483.1
Dataset750	799.38	666.15	579.26	526.6
Dataset800	879.98	733.32	637.67	579.7
Dataset850	961.34	801.12	696.63	633.3
Dataset900	1027.38	856.15	744.48	676.8
Dataset950	1106.02	921.68	801.46	728.6
Dataset1000	1160.82	967.35	841.17	764.7

E. RELIABILITY

Reliability is an important metric in fog, which measures the ability of a system to consistently perform its functions without failure. Ensuring high reliability is essential for mission-critical applications, where task failures can lead to significant operational disruptions. A reliable task scheduling algorithm guarantees consistent performance even in dynamic and resource-constrained environments. This study evaluated reliability using the experimental configuration described in Table 3. The datasets were categorized into small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), which represent different workload complexities and volumes. For small datasets, DDPG achieved significantly higher reliability scores compared to baseline algorithms such as DQN, A2C, and A3C, as shown in Table 16. The reliability values of DDPG ranged from 0.97 to 0.98, while A3C and A2C achieved values between 0.85 and 0.88, and DQN achieved values between 0.68 and 0.70. These results are visualized in Figure 16. For medium datasets, the results illustrated in Table 17 show that DDPG continued its superior performance, achieving reliability scores of 0.98 to 0.99.

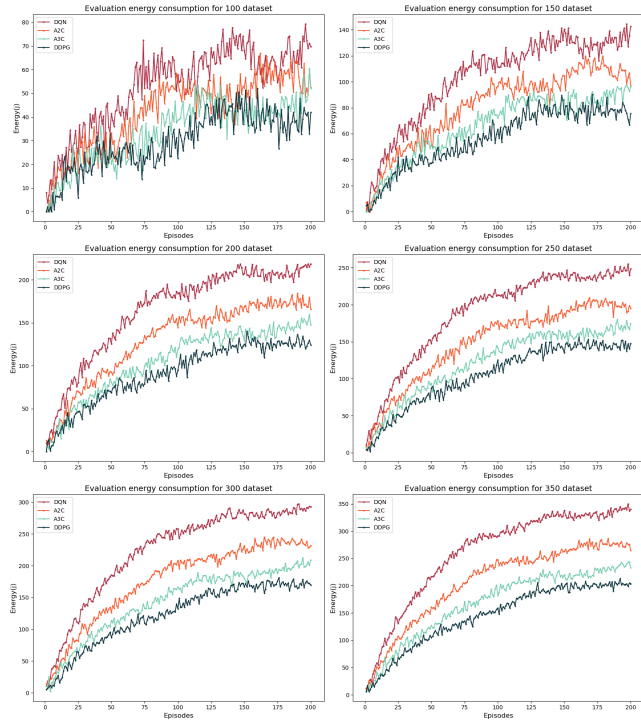


FIGURE 13: Energy Consumption of 100_350 datasets

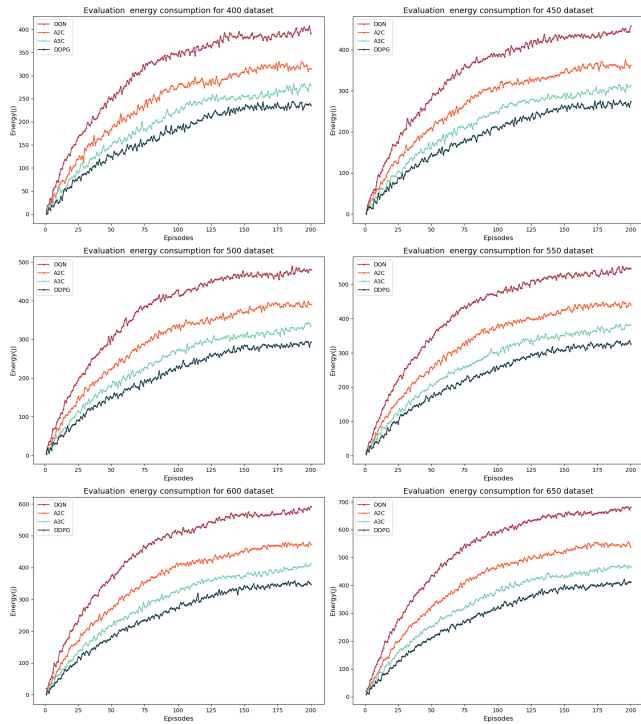


FIGURE 14: Energy Consumption of 400_650 datasets

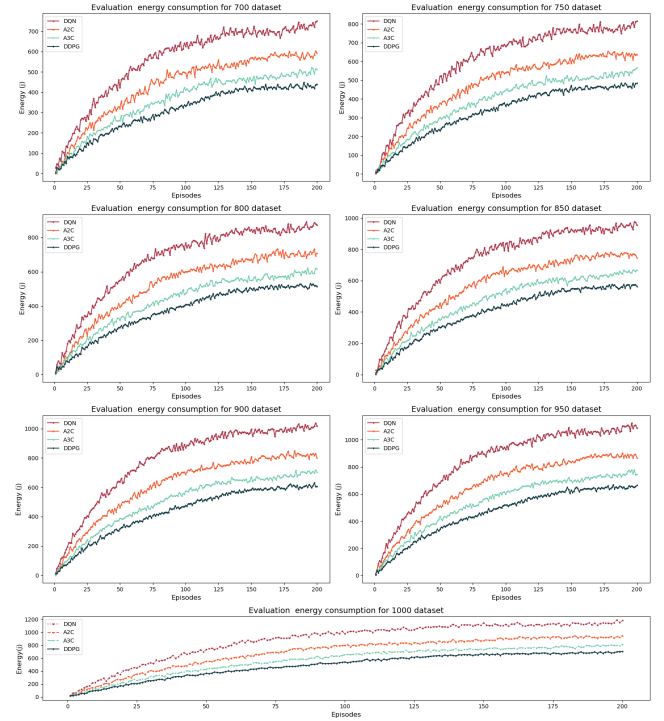


FIGURE 15: Energy Consumption of 700_1000 datasets

This represents a significant improvement over A3C, which achieved values between 0.89 and 0.90, and A2C, which achieved values between 0.78 and 0.88. These trends are illustrated in Figure 17. In the case of large datasets, as summarized in Table 18, DDPG demonstrated exceptional reliability, with scores ranging from 0.99 to 1.00. This is significantly ahead of A3C, which achieved values of 0.88 to 0.90, and A2C, which achieved values of 0.79 to 0.89. These results are visually represented in Figure 18. The experimental results confirm the robustness of DDPG in maintaining high reliability across all dataset categories. The ability to dynamically adjust to changing conditions and efficiently handle tasks ensures consistent performance even in complex fog computing scenarios. This establishes DDPG as a reliable solution for task scheduling in distributed fog environments.

TABLE 16: Reliability for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	0.689	0.731	0.855	0.978
Dataset150	0.687	0.798	0.856	0.976
Dataset200	0.698	0.779	0.854	0.977
Dataset250	0.696	0.798	0.821	0.982
Dataset300	0.695	0.768	0.823	0.983
Dataset350	0.696	0.794	0.832	0.982

F. FAULT

Fault tolerance is a key metric in fog computing systems, as it determines the system's ability to handle failures while performing tasks without interruption. A fault-tolerant task

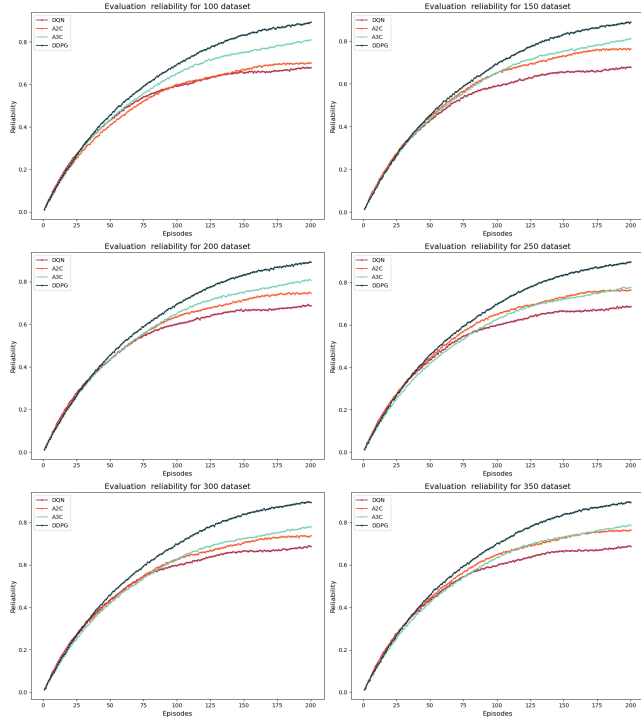


FIGURE 16: Reliability of 100_350 datasets

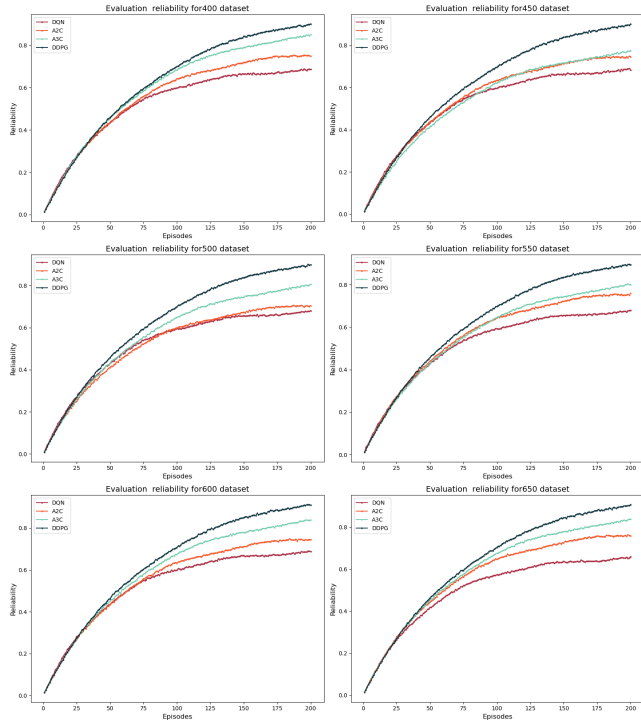


FIGURE 17: Reliability of 400_650 datasets

TABLE 17: Reliability for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	0.696	0.784	0.898	0.986
Dataset450	0.695	0.778	0.815	0.983
Dataset500	0.687	0.734	0.849	0.985
Dataset550	0.686	0.787	0.846	0.981
Dataset600	0.698	0.778	0.887	0.999
Dataset650	0.665	0.794	0.885	0.991

TABLE 18: Reliability Metrics for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	0.687	0.748	0.889	0.996
Dataset750	0.698	0.787	0.887	0.997
Dataset800	0.696	0.785	0.898	0.998
Dataset850	0.689	0.779	0.886	0.992
Dataset900	0.698	0.787	0.884	0.996
Dataset950	0.645	0.798	0.865	0.998
Dataset1000	0.656	0.798	0.884	0.995

scheduling algorithm dynamically reallocates tasks to available resources, reducing the impact of failures and maintaining stable system performance. In this study, fault tolerance was evaluated using the experimental setup described in Table 3. The datasets were categorized into small (100-350 tasks), medium (400-650 tasks), and large (700-1000 tasks), representing different levels of task complexity and system stress. For small datasets, DDPG demonstrated superior fault tolerance compared to baseline algorithms such as DQN, A2C, and A3C, as shown in Table 19. The fault tolerance values for DDPG range from 38.9 to 52.4 units, compared to A3C (42.8 to 57.6 units), A2C (49.2 to 66.3 units), and DQN (59.0 to 79.6 units). These results are visualized in Figure 19. On medium datasets, DDPG continues to show strong performance, as shown in Table 20. The fault tolerance values for DDPG range from 27.3 to 36.8 units, for A3C from 30.0 to 40.4 units, for A2C from 34.5 to 46.6 units, and for DQN from 41.4 to 55.9 units. These results are illustrated in Figure 20. For large datasets, as summarized in Table 21, DDPG demonstrated exceptional fault tolerance, with values ranging from 13.5 to 23.3 units. This is significantly ahead of A3C (15.8 to 25.6 units), A2C (18.2 to 29.4 units), and DQN (20.5 to 35.3 units). These results are illustrated visually in Figure 21. Experimental results show that DDPG effectively enhances fault tolerance by dynamically reassigning tasks in response to failures and ensuring continuity of operations. This makes DDPG a highly reliable solution for managing tasks in dynamic and fault-prone fog computing environments.

VI. CONCLUSION

This research proposes a novel adaptive scheduling framework using the DDPG algorithm, which deals well with continuous action spaces in dynamic environments such as the fog computing domain. In IoT applications, the unpredictability of task demands and resource availability make it a particularly valuable approach. In this context, the adap-

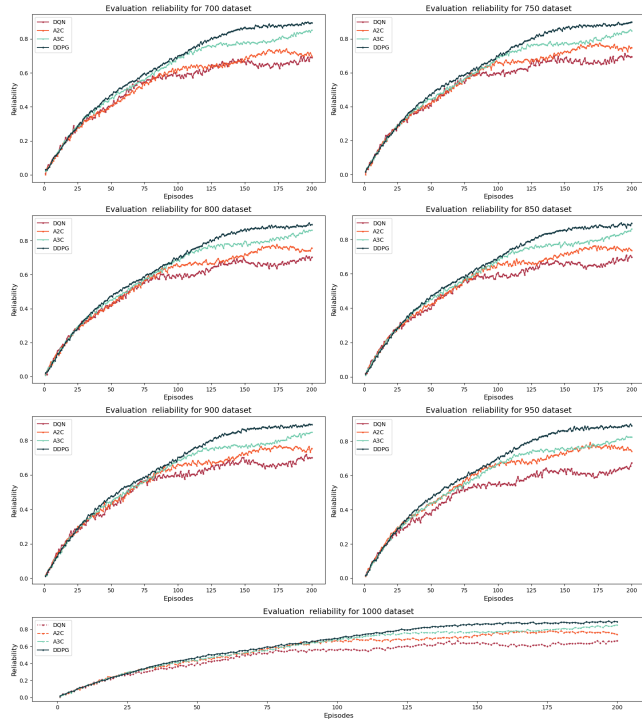


FIGURE 18: Reliability of 700_1000 datasets

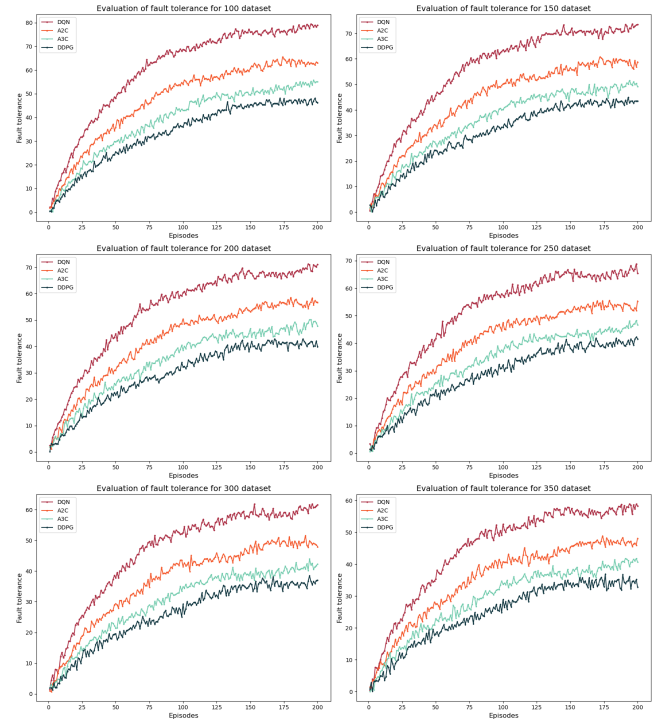


FIGURE 19: Fault tolerance of 100_350 datasets

TABLE 19: Fault Tolerance for Small Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset100	79.55	66.29	57.64	52.4
Dataset150	73.62	61.35	53.35	48.5
Dataset200	70.44	58.7	51.04	46.4
Dataset250	67.7	56.42	49.06	44.6
Dataset300	61.33	51.11	44.44	40.4
Dataset350	59.05	49.21	42.79	38.9

TABLE 20: Fault Tolerance for Medium Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset400	55.86	46.55	40.48	36.8
Dataset450	52.37	43.64	37.95	34.5
Dataset500	51.86	42.5	36.96	33.6
Dataset550	47.66	39.72	34.54	31.4
Dataset600	45.08	37.57	32.67	29.7
Dataset650	41.44	34.53	30.03	27.3

TABLE 21: Fault Tolerance for Large Dataset

Dataset	DQN	A2C	A3C	DDPG
Dataset700	35.36	29.47	25.63	23.3
Dataset750	32.94	27.45	23.87	21.7
Dataset800	28.08	23.4	20.35	18.5
Dataset850	26.41	22.01	19.14	17.4
Dataset900	23.68	19.73	17.16	15.6
Dataset950	21.86	18.22	15.84	14.4
Dataset1000	20.5	17.08	14.85	13.5

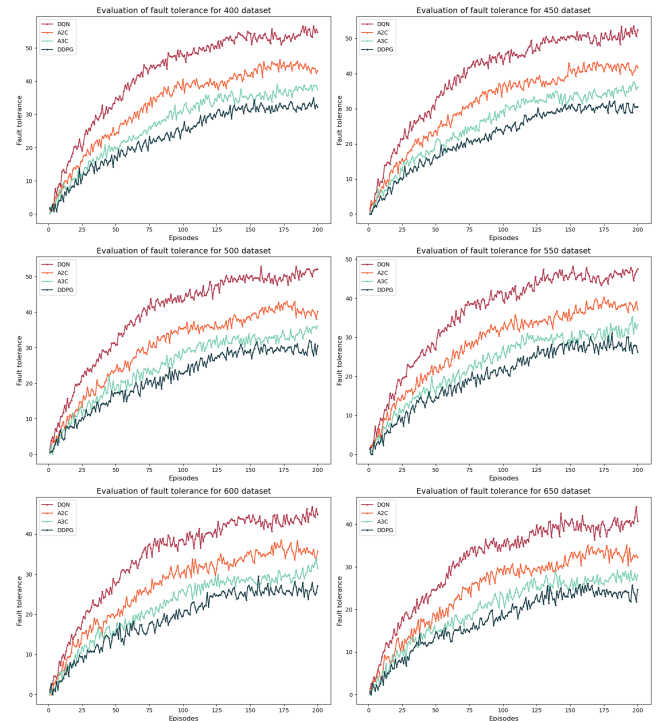


FIGURE 20: Fault tolerance of 400_650 datasets

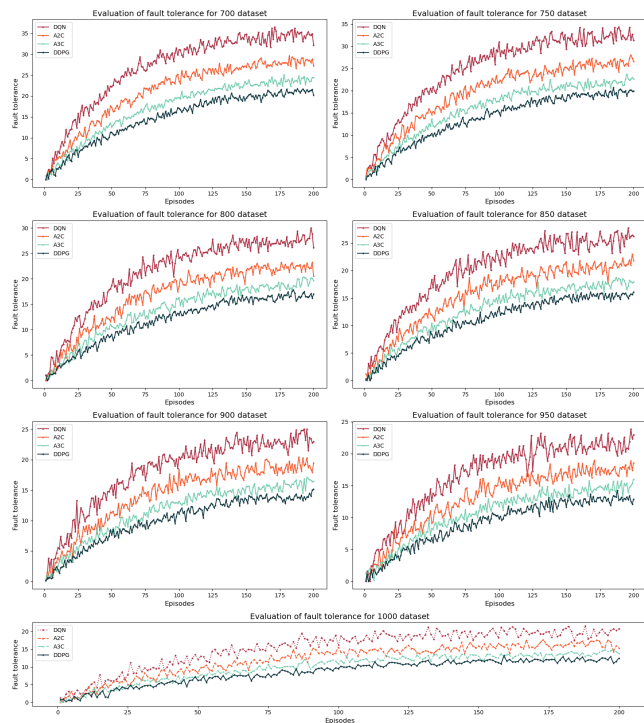


FIGURE 21: Fault tolerance of 700_1000 datasets

tive framework of the DDPG algorithm allows for dynamic tracking of real-time changes, further improving the range of metrics that are of particular importance in latency-sensitive applications. One of the key advantages of this framework is that it improves makespan, fault tolerance, scalability, and reliability, thereby increasing overall system performance by 20% to 30%. For example, the reduction in makespan and improved fault tolerance ensure that critical tasks are completed efficiently and reliably even under challenging conditions. Moreover, this framework reduces bandwidth and latency requirements by up to 40% by processing data close to its source, and significantly improves data privacy and management efficiency. Verification of this framework through simulations in a simple environment using Google Cloud Jobs datasets confirms its effectiveness. It not only shows theoretical progress in the adaptive scheduling domain but also brings significant practical benefits, making it a robust solution for real-world IoT applications that demand high reliability and efficient data processing. The integration of advanced machine learning techniques in fog computing will revolutionize how tasks are managed and executed in distributed computing environments.

REFERENCES

- [1] R. Sandhu, M. Faiz, H. Kaur, A. Srivastava, and V. Narayan, "Enhancement in performance of cloud computing task scheduling using optimization strategies," *Cluster Computing*, pp. 1–24, 2024.
- [2] A. Keivani and J.-R. Tapamo, "Task scheduling in cloud computing: A review," in *2019 International conference on advances in big data, computing and data communication systems (icABCD)*. IEEE, 2019, pp. 1–6.

- [3] S. B. Kamtam, Q. Lu, F. Bouali, O. C. Haas, and S. Birrell, "Network latency in teleoperation of connected and autonomous vehicles: A review of trends, challenges, and mitigation strategies," *Sensors*, vol. 24, no. 12, p. 3957, 2024.
- [4] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, "Towards low-latency service delivery in a continuum of virtual resources: State-of-the-art and research directions," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2557–2589, 2021.
- [5] N. A. Angel, D. Ravindran, P. D. R. Vincent, K. Srinivasan, and Y.-C. Hu, "Recent advances in evolving computing paradigms: Cloud, edge, and fog technologies," *Sensors*, vol. 22, no. 1, p. 196, 2021.
- [6] H. Kuchuk and E. Malokhvii, "Integration of iot with cloud, fog, and edge computing: A review," *Advanced Information Systems*, vol. 8, no. 2, pp. 65–78, 2024.
- [7] B. Omoniwa, R. Hussain, M. A. Javed, S. H. Bouk, and S. A. Malik, "Fog/edge computing-based iot (feciot): Architecture, applications, and research issues," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4118–4149, 2018.
- [8] L. Zhao, "Privacy-preserving distributed analytics in fog-enabled iot systems," *Sensors*, vol. 20, no. 21, p. 6153, 2020.
- [9] A. Munir, J. Kwon, J. H. Lee, J. Kong, E. Blasch, A. J. Aved, and K. Muhammad, "Fogsurv: A fog-assisted architecture for urban surveillance using artificial intelligence and data fusion," *IEEE Access*, vol. 9, pp. 111 938–111 959, 2021.
- [10] Z. Movahedi, B. Defude, and A. M. Hosseininia, "An efficient population-based multi-objective task scheduling approach in fog computing systems," *Journal of Cloud Computing*, vol. 10, no. 1, p. 53, 2021.
- [11] Y. Sun, F. Lin, and H. Xu, "Multi-objective optimization of resource scheduling in fog computing using an improved nsga-ii," *Wireless Personal Communications*, vol. 102, pp. 1369–1385, 2018.
- [12] M. A. Ibrahim and S. Askar, "An intelligent scheduling strategy in fog computing system based on multi-objective deep reinforcement learning algorithm," *IEEE Access*, vol. 11, pp. 133 607–133 622, 2023.
- [13] P. Choppa and S. Mangalampalli, "An efficient deep reinforcement learning based task scheduler in cloud-fog environment," *Cluster Computing*, vol. 28, no. 1, pp. 1–26, 2025.
- [14] P. Choppa and S. S. Mangalampalli, "A hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning," *IEEE Access*, 2024.
- [15] S. Cao, Z. Zhan, C. Dai, S. Chen, W. Zhang, and Z. Han, "Delay-aware and energy-efficient iot task scheduling algorithm with double blockchain enabled in cloud-fog collaborative networks," *IEEE Internet of Things Journal*, 2023.
- [16] A. Jangra and N. Mangla, "An efficient load balancing framework for deploying resource scheduling in cloud based communication in healthcare," *Measurement: Sensors*, vol. 25, p. 100584, 2023.
- [17] S. Seifhosseini, M. H. Shirvani, and Y. Ramzanpoor, "Multi-objective cost-aware bag-of-tasks scheduling optimization model for iot applications running on heterogeneous fog environment," *Computer Networks*, vol. 240, p. 110161, 2024.
- [18] B. M. Nguyen, T. Nguyen, Q.-H. Vu, H. H. Tran, H. Vo, H. T. T. Binh, S. Yu, Z. Wu et al., "A novel nature-inspired algorithm for optimal task scheduling in fog-cloud blockchain system," *IEEE Internet of Things Journal*, 2023.
- [19] M. Abdel-Basset, R. Mohamed, K. M. Sallam, and I. M. Hezam, "Multi-objective task scheduling method for cyber-physical-social systems in fog computing," *Knowledge-Based Systems*, vol. 280, p. 111009, 2023.
- [20] K. Anusha, S. K. Medishetti, P. Archana, G. R. Karri, and P. Kuppusamy, "Mao: An efficient resource utilization of task scheduling in cloud fog environment," in *2024 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET)*. IEEE, 2024, pp. 1–8.
- [21] P. Zhang, N. Chen, G. Xu, N. Kumar, A. Barnawi, M. Guizani, Y. Duan, and K. Yu, "Multi-target-aware dynamic resource scheduling for cloud-fog-edge multi-tier computing network," *IEEE Transactions on Intelligent Transportation Systems*, 2023.
- [22] A. Najafizadeh, A. Salajegheh, A. M. Rahmani, and A. Sahafi, "Multi-objective task scheduling in cloud-fog computing using goal programming approach," *Cluster Computing*, vol. 25, no. 1, pp. 141–165, 2022.
- [23] J. Z. Ahmadabadi, S. E. Mood, and A. Souiri, "Star-quake: A new operator in multi-objective gravitational search algorithm for task scheduling in iot based cloud-fog computing system," *IEEE Transactions on Consumer Electronics*, 2023.

- [24] V. Jain and B. Kumar, "Qos-aware task offloading in fog environment using multi-agent deep reinforcement learning," *Journal of Network and Systems Management*, vol. 31, no. 1, p. 7, 2023.
- [25] G. Goel and R. Tiwari, "Igwoa: Improved grey wolf optimization algorithm for resource scheduling in cloud-fog environment for delay-sensitive applications," *Peer-to-Peer Networking and Applications*, vol. 17, no. 3, pp. 1768–1790, 2024.
- [26] K. Alataoun, K. Matrouk, M. A. Mohammed, J. Nedoma, R. Martinek, and P. Zmij, "A novel low-latency and energy-efficient task scheduling framework for internet of medical things in an edge fog cloud system," *Sensors*, vol. 22, no. 14, p. 5327, 2022.
- [27] F. M. Talaat, H. A. Ali, M. S. Saraya, and A. I. Saleh, "Effective scheduling algorithm for load balancing in fog environment using cnn and mpso," *Knowledge and Information Systems*, vol. 64, no. 3, pp. 773–797, 2022.
- [28] S. S. Tripathy, M. Rath, N. Tripathy, D. S. Roy, J. S. A. Francis, and S. Beborita, "An intelligent health care system in fog platform with optimized performance," *Sustainability*, vol. 15, no. 3, p. 1862, 2023.
- [29] R. Siyadatzaheh, F. Mehrafrooz, M. Ansari, B. Safaei, M. Shafique, J. Henkel, and A. Ejlali, "Relief: A reinforcement-learning-based real-time task assignment strategy in emerging fault-tolerant fog computing," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 752–10 763, 2023.
- [30] W. Deng, L. Zhu, Y. Shen, C. Zhou, J. Guo, and Y. Cheng, "A novel multi-objective optimized dag task scheduling strategy for fog computing based on container migration mechanism," *Wireless Networks*, pp. 1–15, 2024.
- [31] A. M. Yadav, K. N. Tripathi, and S. Sharma, "An opposition-based hybrid evolutionary approach for task scheduling in fog computing network," *Arabian Journal for Science and Engineering*, vol. 48, no. 2, pp. 1547–1562, 2023.
- [32] R. Ghafari and N. Mansouri, "An efficient task scheduling in fog computing using improved artificial hummingbird algorithm," *Journal of Computational Science*, vol. 74, p. 102152, 2023.
- [33] O. Sharma, G. Rathee, C. A. Kerrache, and J. Herrera-Tapia, "Two-stage optimal task scheduling for smart home environment using fog computing infrastructures," *Applied Sciences*, vol. 13, no. 5, p. 2939, 2023.
- [34] M. Saad, R. N. Enam, and R. Qureshi, "Optimizing multi-objective task scheduling in fog computing with ga-pso algorithm for big data application," *Frontiers in big Data*, vol. 7, p. 1358486, 2024.
- [35] F. A. Saif, R. Latip, Z. M. Hanapi, and K. Shafinah, "Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing," *IEEE Access*, vol. 11, pp. 20 635–20 646, 2023.
- [36] A. Ali, S. A. A. Shah, T. Al Shloul, M. Assam, Y. Yasin, S. Lim, and A. Zia, "Multi-objective harris hawks optimization based task scheduling in cloud-fog computing," *IEEE Internet of Things Journal*, 2024.
- [37] M. Abdel-Basset, N. Moustafa, R. Mohamed, O. M. Elkomy, and M. Abouhawwash, "Multi-objective task scheduling approach for fog computing," *IEEE Access*, vol. 9, pp. 126 988–127 009, 2021.
- [38] T. Yang, Y. Li, J. He, Z. Liu, F. Ren, T. Wang, and G. Hou, "Secure and traceable multikey image retrieval in cloud-assisted internet of things," *IEEE Internet of Things Journal*, 2024.
- [39] Y. Miao, Y. Yang, X. Li, L. Wei, Z. Liu, and R. H. Deng, "Efficient privacy-preserving spatial data query in cloud computing," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 122–136, 2023.
- [40] Y. Miao, F. Li, X. Li, Z. Liu, J. Ning, H. Li, K.-K. R. Choo, and R. H. Deng, "Time-controllable keyword search scheme with efficient revocation in mobile e-health cloud," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3650–3665, 2023.
- [41] M. Ibrahim, Y. Lee, and D.-H. Kim, "Dalbfog: Deadline-aware and load-balanced task scheduling for the internet of things in fog computing," *IEEE Systems, Man, and Cybernetics Magazine*, vol. 10, no. 1, pp. 62–71, 2024.
- [42] X. Tang, F. Liu, B. Wang, D. Xu, J. Jiang, Q. Wu, and C. P. Chen, "Workflow scheduling based on asynchronous advantage actor-critic algorithm in multi-cloud environment," *Expert Systems with Applications*, vol. 258, p. 125245, 2024.
- [43] P. Singh, M. Dutta, and N. Aggarwal, "A review of task scheduling based on meta-heuristics approach in cloud computing," *Knowledge and Information Systems*, vol. 52, pp. 1–51, 2017.
- [44] A. Hussain and M. Aleem, "Gocj: Google cloud jobs dataset for distributed and cloud computing infrastructures," *Data*, vol. 3, no. 4, p. 38, 2018.



PRASHANTH CHOPPARA received B.Tech from Acharya Nagarjuna University, in 2018 and the M.Tech degree from JNTUK University, in 2021. He is present pursuing Ph.D.in School of Computer Science and Engineering (SCOPE) VIT – AP University, Andhra Pradesh, India. His area of interests in research are Cloud computing, Fog computing, Machine learning, deep Learning, Reinforcement Learning.



S. SUDHEER MANGALAMPALLI (Member, IEEE) is currently an Associate Professor with the Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education known as MAHE (Institution of Eminence, Deemed to be University). He is a passionate Researcher. Toward his profile, he has 35 publications, which are indexed in Scopus and SCI databases. He authored three textbooks. His research interests include scheduling in cloud computing, edge computing, and fog computing. He is a member of The Institution of Engineers. He is an active reviewer of various SCI-indexed journals.

...